

Trabajo Práctico N°1:

“ arVault ”

75.73 / TB034 - Arquitectura de Software
Segundo cuatrimestre de 2026

Integrantes: QA-lity Crew		
Apellido, Nombres	N° Padrón	Email
Giménez, Tomás	110166	togimenez@fi.uba.ar
Corn, Franco	109025	fcorn@fi.uba.ar
Ruiz Sugliani, Santiago Nahuel	106768	sruizs@fi.uba.ar
Zajic Gisela Daiana	108735	gzajic@fi.uba.ar

1. Introducción

La startup **arVault** es una fintech que opera una billetera digital orientada al cambio de divisas, cuyo principal diferencial competitivo es ofrecer la tasa de cambio en tiempo real más conveniente del mercado, sin gap entre compra y venta. En sus inicios, el backend fue desarrollado bajo la premisa "fake it until you make it", priorizando la entrega rápida de funcionalidad por sobre la solidez arquitectónica para captar inversores tempranos.

Tras el crecimiento inicial de usuarios y la incorporación de cuentas multimonedas respaldadas en entidades bancarias, comenzaron a manifestarse problemas de degradación de servicio, inconsistencias de datos y reclamos de clientes. Frente a este escenario, arVault contrató a nuestro equipo de arquitectos para realizar una auditoría integral del sistema y proponer e implementar mejoras arquitectónicas concretas.

Este informe documenta el diagnóstico y rediseño del servicio. En primer lugar, se analiza la arquitectura, el código y la infraestructura recibidos, identificando y justificando los Atributos de Calidad (QAs) críticos para el negocio frente a los secundarios. En segundo lugar, se instrumentan métricas de telemetría de negocio sobre StatsD/Grafana y se ejecutan pruebas de carga con Artillery para caracterizar empíricamente el comportamiento del caso base. Finalmente, se proponen e implementan tácticas arquitectónicas concretas orientadas a resolver las limitaciones detectadas, evaluando su impacto mensurable y los trade-offs asociados.

2. Análisis y Vista de la Arquitectura Inicial (Caso Base)

2.1. Descripción del Sistema y Diagnóstico Inicial

El sistema corre íntegramente en un único servidor orquestado con Docker Compose. El cliente se conecta vía HTTP al puerto 5555, donde Nginx actúa como reverse proxy redirigiendo cada request a la única instancia de la API Node.js en el puerto 3000, sin balanceo de carga ni redundancia. La persistencia del estado se realiza sobre archivos .json en disco local de forma periódica según una configuración establecida al momento de iniciar el servicio; este no cuenta con base de datos externa ni almacenamiento compartido (toda la información se mantiene en memoria y se persiste en disco de forma periódica).

Al iniciar el servicio, se carga en memoria mediante la función `init()` el estado de accounts, rates y log a partir del contenido almacenado en sus respectivos archivos (ubicados en `state/<archivo>.json`). Posteriormente, para cada uno de ellos se establece un timer para guardar en disco los valores en memoria. Esta persistencia se ejecuta cada 1 segundo para accounts y log, mientras que para rates se ejecuta cada 5 segundos.

Además, la función `save()` escribe directamente sobre los archivos destino mediante `fs.writeFile` sin operaciones atómicas (como escribir en un archivo temporal y renombrar) ni journaling. Si el proceso se interrumpe abruptamente durante el guardado (crash del contenedor o corte de energía), el archivo .json puede quedar truncado o corrupto con 0 bytes, impidiendo que el servicio vuelva a iniciar.

Una vez cargado en memoria el estado según los archivos .json se toma el valor de cada variable en memoria según la solicitud realizada por el cliente.

Se exponen 3 endpoints GET (`/accounts`, `/rates` y `/log`), cada uno encargado de retornar el estado almacenado en memoria. En estos endpoints de lectura, el principal desafío reside en la **Performance y Escalabilidad**: al no implementar una capa de **caché HTTP perimetral** (en Nginx mediante `proxy_cache` o mediante encabezados `Cache-Control/ETag` para el cliente), cada solicitud debe ser resuelta por Node.js. Esto obliga al Event Loop monohilo a serializar repetitivamente los objetos a JSON (`JSON.stringify`), consumiendo ciclos de CPU y ancho de banda innecesariamente ante consultas idénticas. Por su parte, la problemática de **Fiabilidad y consistencia (condiciones de carrera)** queda reservada para las operaciones de escritura como `POST /exchange` y `PUT /accounts/:id/balance`, donde la falta de sincronización y el uso de `await` entre lecturas y modificaciones de saldo permiten inconsistencias financieras bajo tráfico concurrente.

En el caso específico de `GET /log`, el array en memoria crece de forma acumulativa con cada transacción que se realiza (`log.push()`). Como el endpoint no implementa paginación ni límite de registros, el tamaño del payload de respuesta crecerá linealmente con el tiempo, degradando progresivamente el tiempo de respuesta, el uso de memoria y el ancho de banda transferido.

Para el caso de `PUT /accounts/:id/balance` se valida recibir tanto **id** como **balance** (únicamente se comprueba la presencia de estos datos, sin embargo, no se lleva a cabo validación sobre sus tipos o valores negativos). Se iteran todas las entradas en `accounts` hasta encontrar la correspondiente al **id** de la petición y se actualiza su saldo (búsqueda lineal $O(N)$). Finalmente se devuelve la lista completa de cuentas en lugar de responder únicamente la cuenta modificada, lo que genera un tamaño de payload innecesariamente grande a medida que aumenta la cantidad de cuentas.

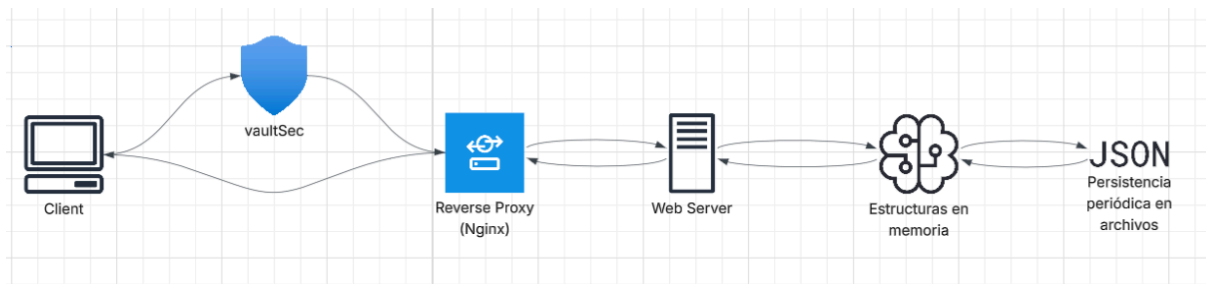
Para el `PUT /rates` se valida la recepción de **baseCurrency**, **counterCurrency** y **rate**, actualizando la cotización en ambas direcciones. Nuevamente se detecta la ausencia de validación sobre los tipos de parámetros recibidos.

Finalmente `POST /exchange` se valida haber recibido **baseCurrency**, **counterCurrency**, **baseAccountid**, **counterAccountid**, **baseAmount** para invocar a `exchange()`. Esta función asíncrona calcula la conversión y busca las cuentas correspondientes mediante iteraciones lineales $O(N)$. Luego, realiza dos transferencias secuenciales invocando a la función asíncrona `transfer()`, la cual simula un servicio externo con una latencia aleatoria de entre 200 y 400 ms por llamada. A nivel de tiempos de respuesta, esto impone un piso de latencia estructural muy alto: cada operación exitosa acumula entre 400 ms y 800 ms solo en esperas de red simuladas. Si la segunda transferencia falla y debe revertirse con una llamada de compensación, la latencia total asciende a ~1200 ms.

Cabe destacar que el endpoint posee varias falencias. Si falla la transferencia que se hace en forma de “rollback” la cuenta origen quedará inconsistente ya que el monto debitado nunca será recuperado y no hay visibilidad sobre ello actualmente. Además “confiar” en que el componente de UI validará que la cuenta origen tenga el saldo necesario abre la puerta a potenciales operaciones inválidas sobre el sistema. Otro problema que tiene el modelo actual es en la consistencia de los datos, se podría dar un escenario en el que mientras se está procesando una operación del tipo **exchange** aparezca otra del tipo **actualizar tarifas** generando una pérdida ya sea para el cliente o para la misma cuenta interna. Por ejemplo, ante un escenario de alta volatilidad:

- **Estado inicial:** Cotización USD/ARS = 10. Saldo Cliente A: USD 200 / ARS 0. Cuenta interna de la bóveda: ARS 2000 / USD 0.
- **T0:** El cliente A solicita convertir USD 200 a ARS (esperando recibir ARS 2000).
- **T1:** Se ejecuta la primera transferencia: se debitan USD 200 del cliente y se acreditan en la cuenta interna.
- **T2:** En paralelo, ingresa un `PUT /rates` que actualiza bruscamente la cotización a USD/ARS = 1.
- **T3:** Se ejecuta la segunda transferencia acreditando ARS 2000 al cliente A según la cotización original.
- **T4 (Inconsistencia de negocio):** Con la nueva cotización (1:1), los ARS 2000 del cliente equivalen ahora a USD 2000. El cliente multiplicó su capital por 10 habiendo entregado solo USD 200, provocando un grave descalce financiero y pérdida patrimonial para arVault al no haber bloqueado/garantizado la tasa al inicio de la operación.

2.2. Vista de Componentes y Conectores (C&C)



La vista de Componentes y Conectores (C&C) describe formalmente los elementos computacionales en tiempo de ejecución representados en el diagrama anterior, así como los protocolos mediante los cuales interactúan.

Catálogo de Componentes:

1. Client / Traffic Generator (Artillery / HTTP Client):

- **Tipo:** Componente externo / Generador de tráfico HTTP.
- **Responsabilidad:** Emite las peticiones HTTP (consultas de saldo, cotizaciones y transferencias de divisas) hacia la entrada del sistema.

2. vaultSec:

- **Tipo:** Componente de seguridad (no se encuentra implementado sino que se asume su existencia a fines del trabajo práctico).
- **Responsabilidad:** Autenticación y autorización de las solicitudes.

3. Reverse Proxy (Nginx):

- **Tipo:** Servidor web / Proxy inverso (Contenedor Docker `nginx`).
- **Responsabilidad:** Punto único de acceso público al sistema. Recibe las peticiones del Cliente y las redirige hacia la API de la aplicación.

4. Application Server (Node.js API):

- **Tipo:** Servicio de aplicación principal (Contenedor Docker `exchange-api-1`).
- **Responsabilidad:** Ejecuta la lógica de negocio en memoria (gestión de cuentas, cotizaciones, cálculo de conversiones y ejecución de transferencias).

5. Storage (Archivos .json / State):

- **Tipo:** Almacenamiento local en disco (`state/*.json`).
- **Responsabilidad:** Mantiene la semilla inicial y la persistencia en disco de las cuentas, cotizaciones y logs del sistema.

3. Atributos de Calidad

Dado que es arquitectónicamente imposible optimizar simultáneamente todos los atributos de calidad, debemos priorizar aquellos alineados directamente con los objetivos estratégicos y el modelo de negocio de la compañía. Para arVault, una fintech orientada al intercambio de divisas en tiempo real que busca captar rondas de inversión, la arquitectura debe resolver en primera instancia la operabilidad del servicio y la confianza financiera del usuario.

En este informe, clasificamos los atributos evaluados en dos categorías según su naturaleza y alcance:

1. **Atributos Clave o Primarios** (*Impacto Operativo Directo*): Aquellos de naturaleza operativa que percibe directamente el usuario final durante el uso del servicio (Disponibilidad, Fiabilidad, Performance, Escalabilidad y Seguridad). Su incumplimiento degrada la experiencia en tiempo real, genera inconsistencias de saldo o causa caídas que destruyen la reputación de la plataforma.
2. **Atributos Secundarios** (*Estructurales e Internos*): Aquellos que impactan en la organización del código, la infraestructura y el ciclo de vida del software (Visibilidad, Modificabilidad, Robustez, Testeabilidad, Portabilidad, Simplicidad e Interoperabilidad). Condicionan el costo de mantenimiento, la facilidad de evolución y la capacidad de prueba del backend.

A continuación, se definen y justifican detalladamente los Atributos de Calidad Clave del sistema, seguidos del análisis integral de los Atributos Secundarios.

3.1. Atributos de calidad clave (Primarios)

3.1.1. Disponibilidad (Availability)

El objetivo central es garantizar que los usuarios puedan consultar cotizaciones, revisar sus saldos e intercambiar divisas sin sufrir interrupciones en el servicio, incluso durante momentos de alta demanda operacional. Dado que la disponibilidad mide la capacidad del sistema para limitar el tiempo fuera de servicio, nuestro propósito es estructurar una arquitectura capaz de enmascarar o tolerar fallas de modo que la indisponibilidad acumulada no supere un umbral aceptable (por ejemplo, fijar una meta de 3 horas de caída como máximo en un año calendario, aun en ausencia de un SLA contractual explícito). En la implementación recibida, la disponibilidad se ve amenazada principalmente porque el servidor web Node.js funciona como un nodo único sin réplicas; cualquier caída de la instancia detiene por completo la operación de la plataforma.

3.1.2. Fiabilidad e Integridad de Datos (Reliability)

Se posiciona como el atributo de mayor criticidad para arVault, ya que las cuentas financieras y los movimientos entre divisas deben mantener absoluta consistencia tras cada transacción. El análisis del backend recibido revela una ausencia total de mecanismos de transaccionalidad. La función `exchange()` cede el hilo de ejecución mediante la sentencia `await` en al menos dos oportunidades durante las llamadas asíncronas a `transfer()`. Como el proceso de Node.js opera sobre un único Event Loop, las solicitudes concurrentes enviadas durante picos de volatilidad cambiaria se intercalan entre sí, permitiendo que la aplicación lea saldos que cambian en simultáneo. Esto deriva en condiciones de carrera severas (como débitos dobles o inconsistencias en los balances) que destruyen la confiabilidad del servicio.

3.1.3. Performance (Rendimiento)

La propuesta de valor de arVault requiere responder con rapidez tanto en la consulta de cotizaciones como en la ejecución de cambios de moneda. El rendimiento evalúa el tiempo de respuesta que experimenta el cliente y el tiempo necesario para completar la operación. Si la latencia de endpoints como `GET /rates` o `POST /exchange` se incrementa significativamente bajo carga, la cotización mostrada pierde validez respecto del mercado y la experiencia de uso se deteriora. Mantener percentiles de latencia bajos (especialmente p95 y p99) es clave para evitar cuellos de botella, considerando además que las demoras simuladas en el servicio de transferencias (entre 200 y 400 ms por llamada) imponen un piso estructural de latencia de entre 400 y 800 ms por operación exitosa (dos órdenes de magnitud mayor frente a los 1–5 ms observados en los endpoints de lectura), el cual la aplicación debe gestionar bajo concurrencia sin degradar el Event Loop.

La performance del sistema debe analizarse desde diferentes lugares.

- Para los endpoints de lectura podemos decir que la arquitectura actual se destaca positivamente en aspectos como **percepción del usuario** (latencia por ejemplo), **red** y **eficiencia** debido a que la latencia es reducida considerablemente al utilizar los datos en memoria y no hacer llamadas a servicios externos. Por otro lado el ancho de banda al no tener paginación y el costo de convertir los objetos en memoria a json podrían afectar a la eficiencia y red; además del crecimiento inesperado de la memoria utilizada.
- En cuanto a los endpoints de escritura la cosa cambia, los endpoints de `PUT /accounts/:id/balance` y `/rates` **percepción del usuario, red y eficiencia** ya que al igual que mencionamos anteriormente con los de lectura no hacen llamadas a servicios externos y aprovechan la información que tienen en memoria (Exchange podría decirse que también ya que aprovecha la info en memoria para consultar el saldo de la cuenta interna). Por otro lado las potenciales triples llamadas por request al servicio externo de transferencias en el endpoint de exchange afectan a **percepción del usuario y red** negativamente al igual que devolver el payload completo de las cuentas. Recorrer la lista de cuentas en forma lineal afecta a la **eficiencia** también.

3.1.4. Escalabilidad (Scalability)

Resulta indispensable para que arVault pueda absorber picos de tráfico causados por anuncios económicos, volatilidad de mercado o campañas de usuarios, evitando degradaciones severas o caídas que perjudiquen las negociaciones con inversores. En el estado inicial, la API Node.js corre en una sola instancia detrás de Nginx, manteniendo todo el estado en arrays en memoria y persistiéndolo en archivos JSON locales sin un medio compartido. Si se intentara duplicar las instancias del servidor sin resolver la gestión de estado, la consistencia se degradaría aún más que en la situación planteada en Fiabilidad. Las mediciones preliminares sobre GET `/rates` confirman esta limitación estructural, mostrando una pérdida clara de rendimiento a partir de las 350 req/s y fallos masivos por timeout al aproximarse a 500 req/s.

3.1.5. Seguridad (Security)

Evalúa la capacidad de proteger los datos e interacciones frente a accesos no autorizados o usos maliciosos.

El modelo arquitectónico asume la presencia de un componente perimetral (vaultSec) que resuelve la autenticación y autorización inicial del cliente. Sin embargo, la aplicación carece de controles internos: no existe cifrado TLS en la comunicación entre Nginx y la API de Node.js (tráfico en texto plano), no hay mecanismos de control de tasa de solicitudes (Rate Limiting) para prevenir ataques de denegación de servicio (DoS), y endpoints como POST `/exchange` o PUT `/accounts/:id/balance` no realizan validaciones sobre los valores o tipos de datos recibidos.

Si bien la parte de autenticación y autorización está resuelta por el componente perimetral, existen falencias críticas en Integridad, Confidencialidad y No Repudio:

- **Exposición innecesaria de datos (Confidencialidad):** Endpoints como GET `/accounts` y PUT `/accounts/:id/balance` devuelven el listado completo de todas las cuentas del sistema en cada respuesta en lugar de devolver únicamente la cuenta consultada o modificada.
- **Falta de congelamiento de cotización (Integridad):** El endpoint POST `/exchange` no recibe ni valida una referencia temporal de la cotización pactada por el cliente (*quote token*), permitiendo que un cambio simultáneo de cotizaciones altere el monto transferido en pleno procesamiento.
- **Vulnerabilidad ante DoS e Inyecciones:** No existen tácticas perimetrales de Rate Limiting para evitar ataques de denegación de servicio (afectando la Disponibilidad), ni una sanitización estricta de inputs numéricos en los controladores.

3.2. Atributos de calidad secundarios

3.2.1. Visibilidad (Visibility)

Entendida bajo la definición de Fielding como la capacidad de un componente para mediar o monitorear las interacciones entre otros elementos del sistema, la visibilidad en el caso base se encuentra resuelta a nivel de infraestructura.

Nginx actúa como un Reverse Proxy mediador que centraliza el tráfico HTTP de entrada, mientras que el stack compuesto por cAdvisor, StatsD, Graphite y Grafana permite observar el consumo de recursos de los contenedores (CPU y memoria) en tiempo real. No obstante, a nivel de la aplicación Node.js, la visibilidad inicial resultaba nula al no disponer de métricas de negocio (como el volumen operado por moneda o la cantidad de transacciones exitosas versus fallidas), carencia que fue subsanada mediante la instrumentación de métricas UDP a StatsD.

3.2.2. Modificabilidad (Modifiability)

Mide la facilidad y el costo de incorporar cambios o evoluciones en el software.

El código recibido es lo suficientemente compacto como para permitir una comprensión rápida de sus responsabilidades y facilitó definir un rumbo claro para refactorizar la persistencia y la transaccionalidad. Sin embargo, se observa un acoplamiento directo entre el ruteo (`app.js`), la lógica de negocio (`exchange.js`) y la gestión del estado (`state.js`), sin abstracciones como el patrón Repository. A pesar de esto, implementar la interacción con una base de datos externa (como Redis) representa una modificación de complejidad aceptable que permitirá construir una arquitectura más sólida de cara a futuras versiones.

3.2.3. Robustez (Robustness)

Representa la habilidad del sistema para gestionar entradas inválidas o condiciones anómalas sin colapsar ni dejar el estado interno corrupto.

El backend recibido es frágil frente a peticiones malformadas: en `app.js` únicamente se verifica la presencia de los parámetros obligatorios mediante comprobaciones simples (como `if (!baseAmount)`), omitiendo validar si los datos ingresados son numéricos, positivos o de un tipo correcto. Un payload malicioso o erróneo puede provocar excepciones no capturadas en tiempo de ejecución o guardar datos inconsistentes en los arrays de memoria.

3.2.4. Testeabilidad (Testability)

Mide la facilidad con la que el software permite ejecutar pruebas automatizadas para descubrir defectos.

El proyecto carece de una suite de tests unitarios o de integración en su `package.json`, dependiendo exclusivamente de ejecuciones manuales con Postman o scripts de Artillery. Además, el diseño dificulta la incorporación de pruebas automatizadas debido al uso de estado global mutable en memoria (variables `accounts`, `rates` y `log`), el acoplamiento rígido con

`state.js` y la inclusión de una función `transfer()` interna con latencia simulada que no puede ser mockeada ni aislada sin modificar el código fuente.

3.2.5. Portabilidad (Portability)

Es el atributo de calidad mejor resuelto en la solución inicial. Gracias al uso de contenedorización mediante Docker y Docker Compose, todas las dependencias del entorno de ejecución (Node.js, Nginx, Graphite y Grafana) se encuentran encapsuladas en imágenes portables. Esto independiza completamente al software del sistema operativo anfitrión (macOS, Linux o Windows), permitiendo desplegar la infraestructura completa con un único comando ejecutable de forma homogénea.

3.2.6. Simplicidad (Simplicity)

Evalúa la facilidad para comprender e implementar la arquitectura.

En el sistema recibido, la simplicidad fue maximizada de forma deliberada por el fundador bajo la premisa de "fake it until you make it" / KISS, utilizando estructuras de datos nativas en memoria y archivos JSON locales sin capas de abstracción ni bases de datos. El trade-off de esta decisión fue sacrificar atributos críticos como la Fiabilidad, la Concurrencia y la Escalabilidad a cambio de lograr un Time to Market (TTM) mínimo que permitiera mostrar un prototipo funcional a los inversores iniciales.

3.2.7. Interoperabilidad (Interoperability)

Mide la capacidad del sistema para intercambiar información con otros servicios externos.

Hacia las aplicaciones móviles de los usuarios, la API REST sobre HTTP expone contratos JSON estándar. Respecto del servicio de transferencias de fondos, el código actual incluye la función `transfer()` que sólo simula esperas asíncronas de entre 200 y 400 ms devolviendo siempre un valor exitoso, sin protocolos de comunicación real ni gestión de errores bancarios, aunque el flujo de `exchange()` ya contempla la lógica de compensación (rollback). La interoperabilidad con la red bancaria queda simulada, mientras que la integración con la infraestructura de telemetría (StatsD y Grafana) sí se encuentra operativa.

3.2.8. Mantenibilidad (Maintainability)

Evalúa la facilidad con la que los administradores y desarrolladores pueden monitorear, configurar, depurar y mantener el sistema en producción.

En el caso base, Docker Compose simplifica el orquestado inicial, pero la gestión operativa es rígida: modificar variables de entorno o parámetros del servidor exige reconstruir las imágenes de los contenedores. Asimismo, el sistema carece de endpoints de salud (como `/health` o readiness probes) y el manejo de excepciones es elemental, sin registros estructurados de logs ni niveles de trazabilidad (debug, info, error), lo que dificulta la depuración rápida ante fallas no previstas.

4. Pruebas de Carga y Caracterización Inicial

(Caso Base)

Para comprender empíricamente cómo reacciona el backend recibido antes de encarar cualquier cambio en el código o la infraestructura, procedimos a evaluar el comportamiento de los endpoints expuestos mediante la herramienta Artillery. Toda la telemetría generada durante las ejecuciones fue recolectada a través del stack de observabilidad de la solución (StatsD, Graphite y Grafana), registrando la latencia del cliente (mediana y valores máximos), la cantidad de peticiones resueltas o fallidas, y el consumo de recursos de los contenedores.

4.1. Medición de Endpoints de Lectura

4.1.1. Evaluación de la consulta de cotizaciones (GET */rates*)

Para actualizar la medición del caso inicial se probó GET */rates* de forma aislada. El entorno mantuvo una sola instancia de la API, con 1 CPU y 512 MB, NGINX como punto de entrada y los archivos JSON cargados en memoria. Cada usuario virtual realizó una única consulta.

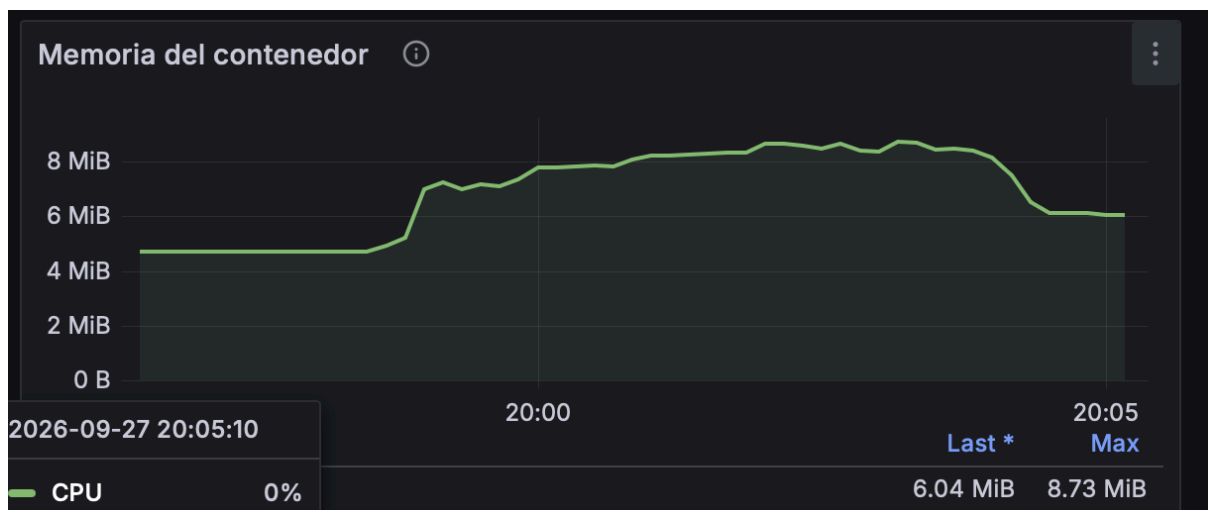
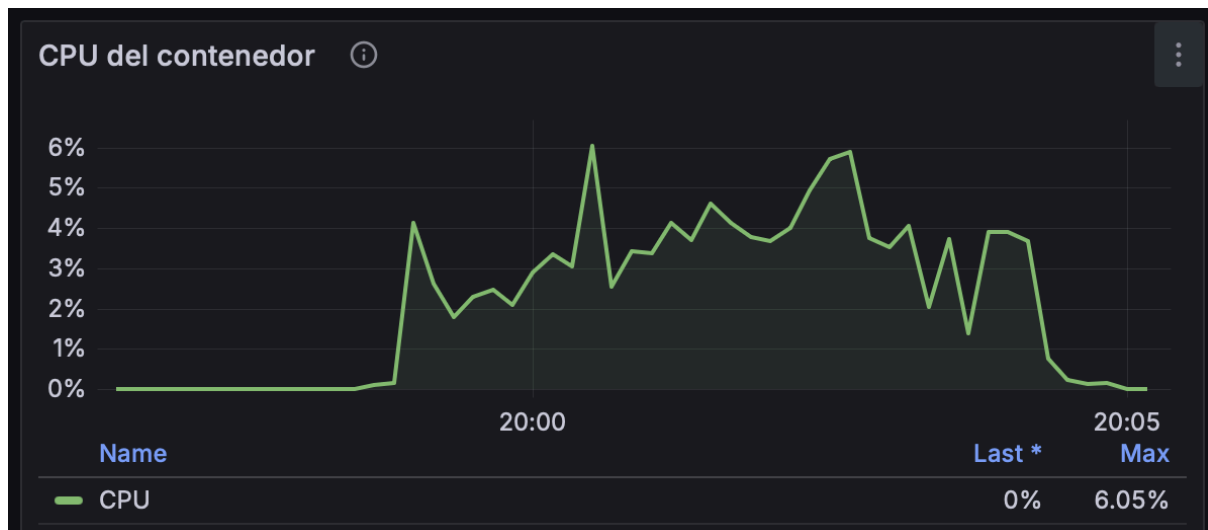
Perfil de carga utilizado:

Etapas	Duración	Carga
Entrada	15 s	100 req/s
Carga media	20 s	1.000 req/s
Carga alta	20 s	2.500 req/s
Máximo	20 s	3.000 req/s
Recuperación	20 s	100 req/s

Resultados:

Métrica	Resultado
Requests enviadas	133.512
Respuestas HTTP 200	133.512
Usuarios fallidos	0
Tiempo medio	0,4 ms
p95	2 ms
p99	4 ms
Máximo	66 ms

El endpoint sostuvo todas las etapas hasta 3.000 req/s sin errores. La latencia se mantuvo estable y, durante la recuperación a 100 req/s, volvió a los valores iniciales. En este escenario no se encontró el punto de quiebre de GET */rates*.

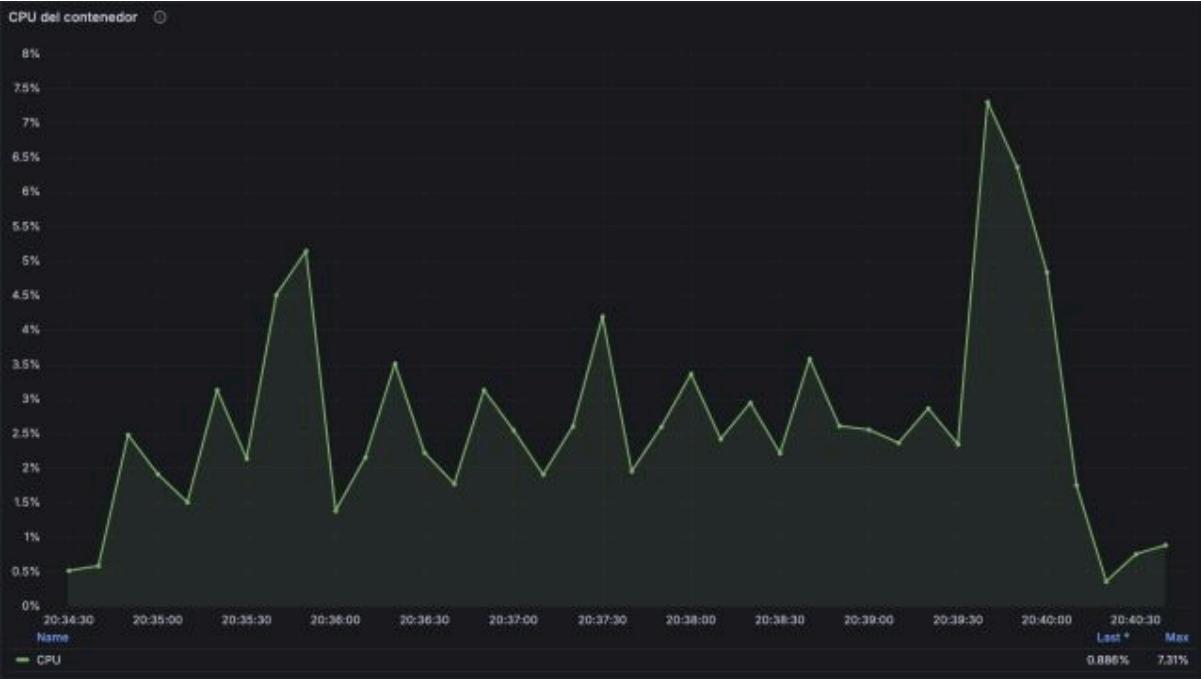
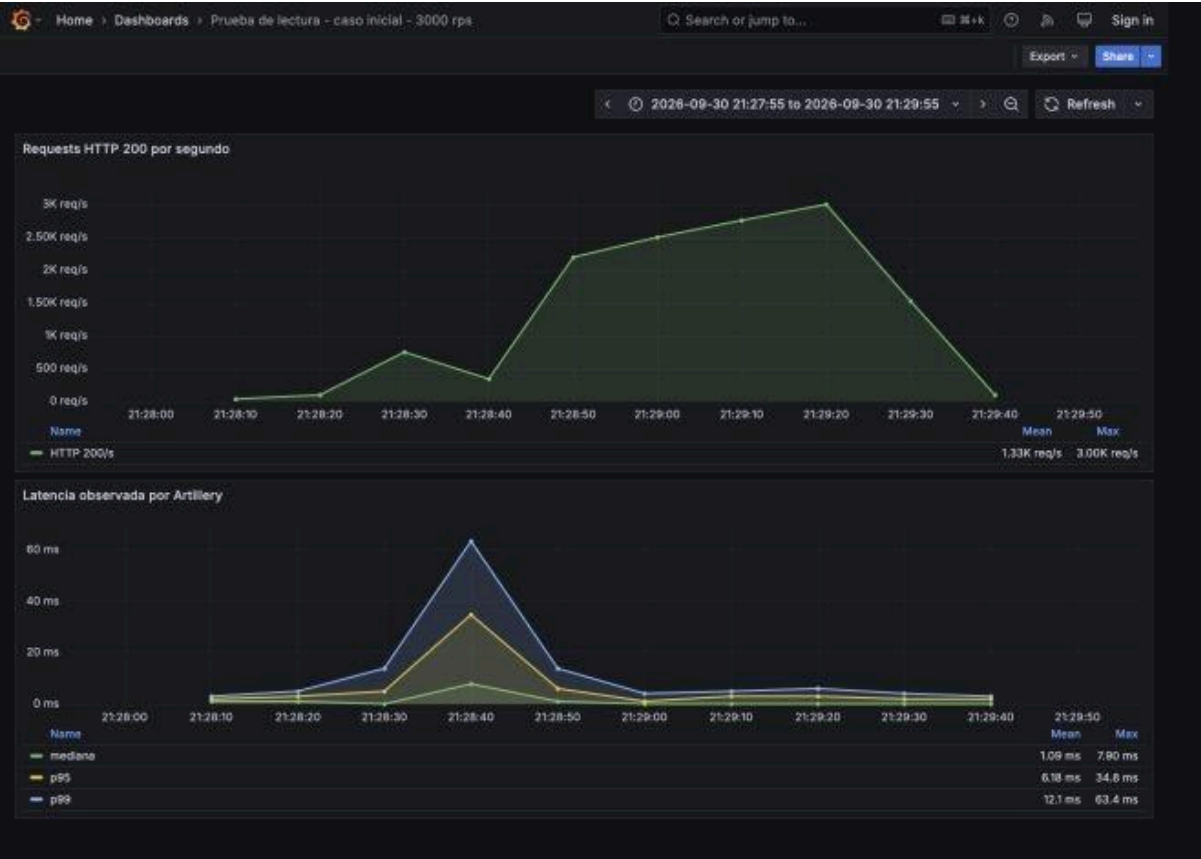


Podemos tomar las siguientes conclusiones:

- El CPU no se ve comprometido por las pruebas (pico máximo de 6,05%), por lo que incluso alcanzando las 3.000 req/s no se observa degradación en el servicio ni saturación de recursos.
- La memoria se mantiene estable y muy lejos del límite (512mb).

4.1.2. Evaluación de la consulta de cuentas (GET /accounts)

Para este caso ejecutaremos las pruebas agregando una variable más, la cantidad de elementos en memoria debido a que es probable que con el incremento de la popularidad del sistema la cantidad de cuentas crezca exponencialmente. Se utilizó el mismo escenario de carga por escalones que en la prueba de cotizaciones evaluado ante diferentes volúmenes de cuentas; la primera fue con la data original. La hipótesis inicial fue que los resultados serían equivalentes, dado que la estructura de datos y la lógica en memoria no difieren significativamente respecto de GET /rates.

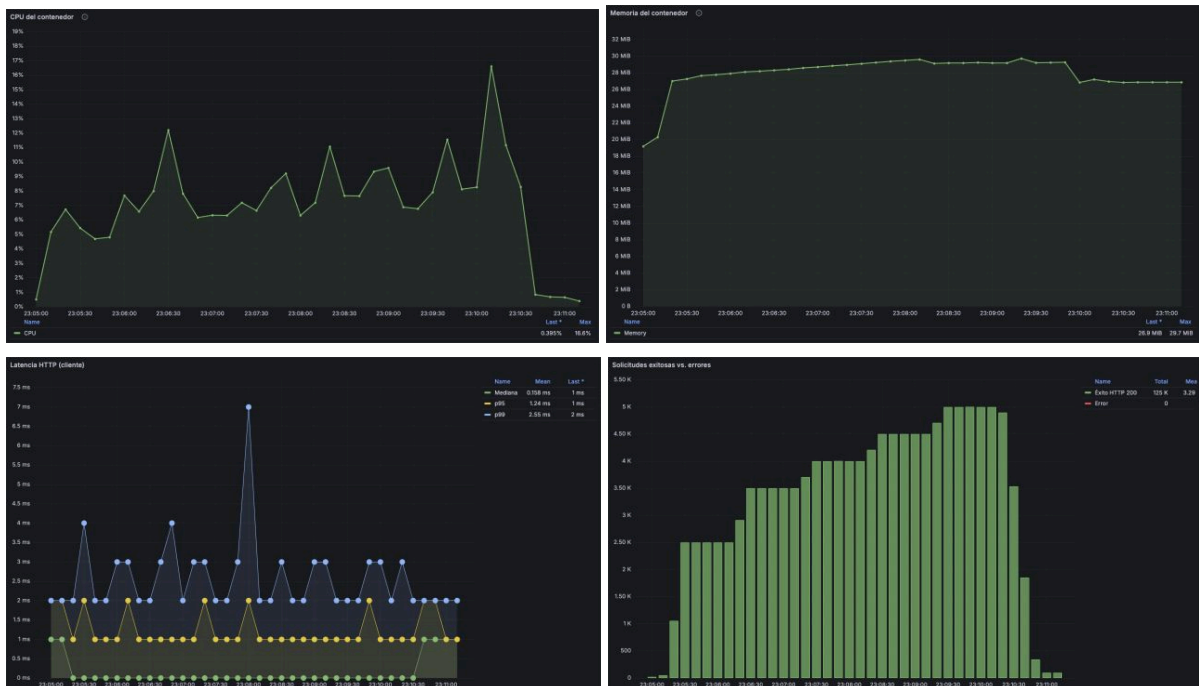




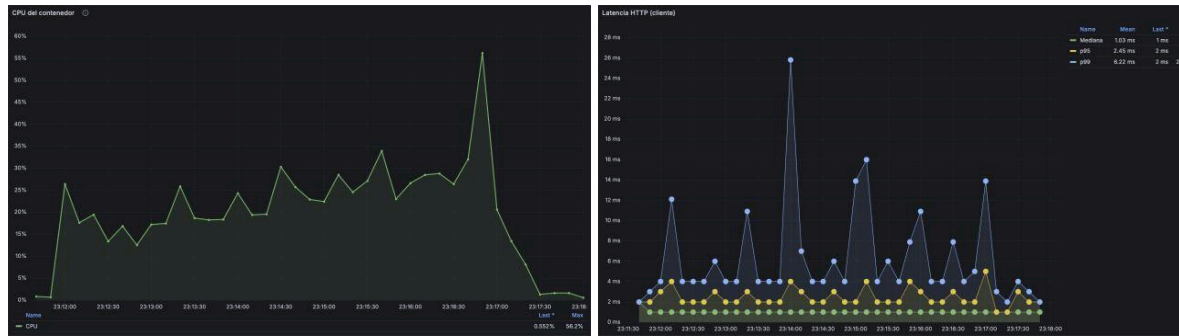
Como se puede observar en las métricas recolectadas, la hipótesis fue validada: el gráfico muestra un comportamiento casi idéntico al de GET /rates, manteniendo estabilidad.

En las siguientes pruebas se fue subiendo la cantidad de elementos pero cambiando el esquema de tráfico a escalones de 60s arrancando con 100 de rps y llegando hasta 500 .

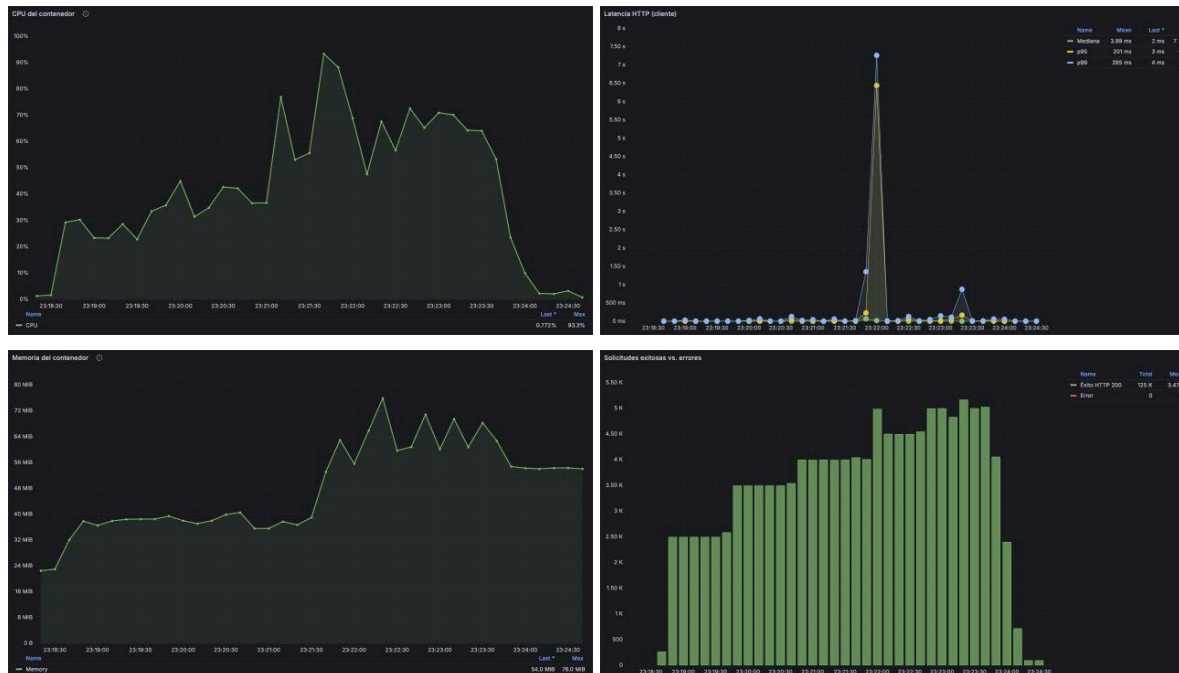
1000 cuentas



5000 cuentas



10000 cuentas



20000 cuentas



Lo que se puede apreciar con las imágenes es que a medida que aumentamos la cantidad de elementos quien se ve más perjudicado es el CPU a diferencia de la memoria que tiene un pico en 100mb aprox.

Cuentas	Respuesta	HTTP 200	Fallos	p95	p99	CPU media	CPU máx.	Memoria media	Memoria máx.
4	0,2 KiB	125.050	0	1 ms	2 ms	2,7 %	7,3 %	34,5 MiB	37,2 MiB
1.000	44,8 KiB	125.050	0	1 ms	3 ms	7,0 %	16,6 %	27,9 MiB	29,7 MiB
5.000	228,4 KiB	125.050	0	3 ms	6 ms	19,8 %	56,2 %	28,7 MiB	31,0 MiB
10.000	457,9 KiB	125.050	0	43,4 ms	262,5 ms	41,3 %	93,3 %	48,1 MiB	76,0 MiB
20.000	926,6 KiB	117.048	8.004	497,8 ms	3.134,5 ms	75,5 %*	100,6 %*	66,0 MiB*	95,8 MiB*

Conclusión:

- Entre 4 y 1.000 cuentas no aparece una degradación.
- Con 5.000 cuentas aumenta el costo de CPU, pero se mantienen baja latencia y cero errores.
- Con 10.000 cuentas comienza la zona crítica: CPU máxima de 93,3 %, p95 de 43,4 ms y p99 de 262,5 ms.
- Con 20.000 cuentas se agota la CPU disponible, aparecen 8.004 timeouts y el p99 supera 3 segundos.
- La memoria nunca se acerca a 512 MiB. El límite observado es CPU, no memoria.

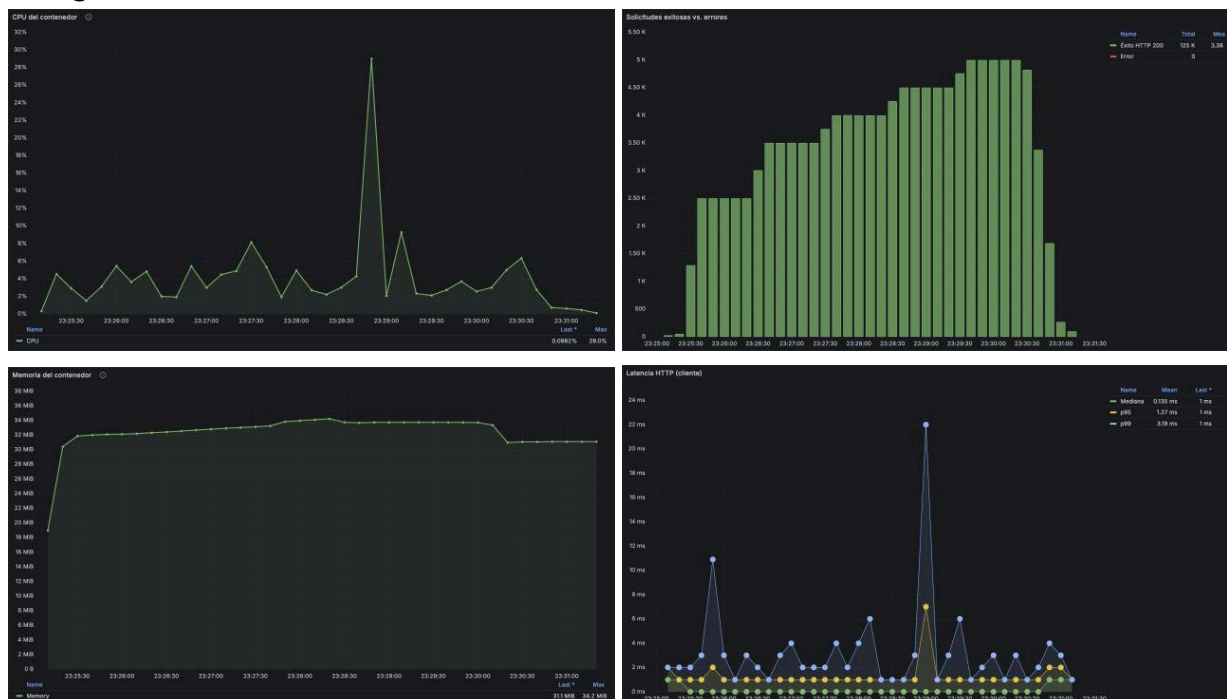
El punto de degradación comienza entre 5.000 y 10.000 cuentas. La pérdida de disponibilidad aparece entre 10.000 y 20.000 cuentas para este perfil de carga.

4.1.3. Evaluación del historial de operaciones (GET /log)

Particularmente, la estructura de log en memoria posee la característica de crecer de manera lineal y acumulativa (orden $O(N)$) en función de la cantidad de transacciones ejecutadas, dado que por cada intercambio exitoso se incorpora un nuevo registro al array.

Siguiendo con la idea de la prueba anterior aplicamos el mismo criterio aquí donde iremos escalando las pruebas hasta llegar a los 5k de logs.

50 logs



500 logs



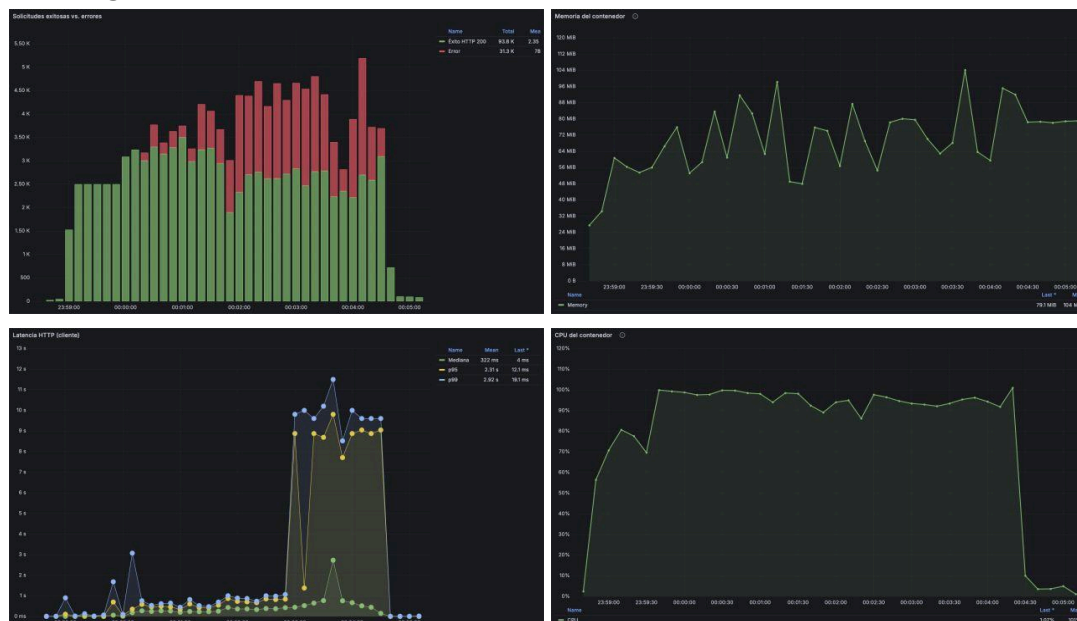
1000 logs



2000 logs



5000 logs



Vemos con las pruebas que los 3 endpoints de lectura al leer en objetos en memoria reaccionan similar ante una carga alta y estable de trabajo en un marco donde la cantidad de elementos no es extremadamente grande. A medida que fue incrementando la cantidad de elementos observamos un patrón similar al que nos encontramos con las cuentas donde cerca de los 5k empezábamos a sentir la degradación, aquí como el objeto es más pesado esto sucede antes entre los 1000 y 2000.

Logs	Respuesta	HTTP 200	Fallos	p95	p99	CPU media	CPU máx.	Memoria media	Memoria máx.
0	2 B	125.050	0	1 ms	3 ms	4,0 %	29,0 %	32,3 MiB	34,2 MiB
100	22,6 KiB	125.050	0	1 ms	3 ms	5,0 %	20,6 %	33,3 MiB	35,7 MiB
500	114,0 KiB	125.050	0	2 ms	5 ms	10,1 %	19,9 %	33,0 MiB	35,3 MiB
1.000	229,7 KiB	125.050	0	3 ms	8,9 ms	18,4 %	36,3 %	29,4 MiB	31,1 MiB
2.000	461,5 KiB	125.050	0	7,9 ms	37 ms	41,1 %	95,5 %	40,0 MiB	46,5 MiB
5.000	1.155,5 KiB	93.844	31.333	5.711,5 ms	9.230,4 ms	78,9 %	101,0 %	69,6 MiB	104,2 MiB

Errores de la variante con 5.000 logs:

- 25.602 ETIMEDOUT.
- 5.731 ECONNRESET.
- Tasa de éxito aproximada: 75,05 %.

Conclusión

- Hasta 1.000 logs el endpoint conserva baja latencia y cero errores.
- Con 2.000 logs aparece la zona crítica: CPU máxima de 95,5 % y crecimiento de la cola de latencia, aunque todavía completa todas las solicitudes.
- Con 5.000 logs la CPU permanece cerca del 100 % durante gran parte de la prueba y se pierde aproximadamente una cuarta parte de las solicitudes.
- La memoria máxima fue 104,2 MiB, todavía muy por debajo del límite de 512 MiB.

El comienzo de la degradación está entre 1.000 y 2.000 logs. La pérdida clara de disponibilidad ocurre entre 2.000 y 5.000 logs.

Hay un patrón claro y es que la serialización de elementos juega un papel importante en cuanto a la performance en las pruebas. Dadas las condiciones iniciales hasta 500 solicitudes por segundo, el sistema procesó de forma estable respuestas de aproximadamente 230 KiB, equivalentes a un caudal nominal de 112 MiB/s. Alrededor de 460 KiB por respuesta, o 225 MiB/s nominales, comenzó la degradación de CPU y latencia. Entre 0,9 y 1,2 MiB por respuesta, equivalentes a 452–564 MiB/s nominales, la CPU se saturó y aparecieron timeouts y conexiones reiniciadas.

Por lo tanto, el umbral no es una cantidad fija de KiB que la CPU pueda procesar: depende del producto entre el tamaño de la respuesta y las solicitudes por segundo. La evidencia permite ubicar el lí

mite práctico de esta configuración entre aproximadamente 112 y 225 MiB/s de JSON serializado; 112 MiB/s fue estable y cerca de 225 MiB/s ya mostró señales críticas

4.2. Medición de Endpoints de Modificación de Estado

4.2.1. Evaluación de la actualización de tasas (PUT /rates)

En este caso la prueba que llevamos a cabo busca simular un determinado momento de mercado donde se realiza una gran cantidad de actualizaciones en el valor entre divisas y posteriormente esta situación “se calma” provocando que el uso del servicio para este endpoint vuelva a valores esperables, de esta forma buscamos poner a prueba al servicio bajo una situación de volatilidad para determinada moneda.

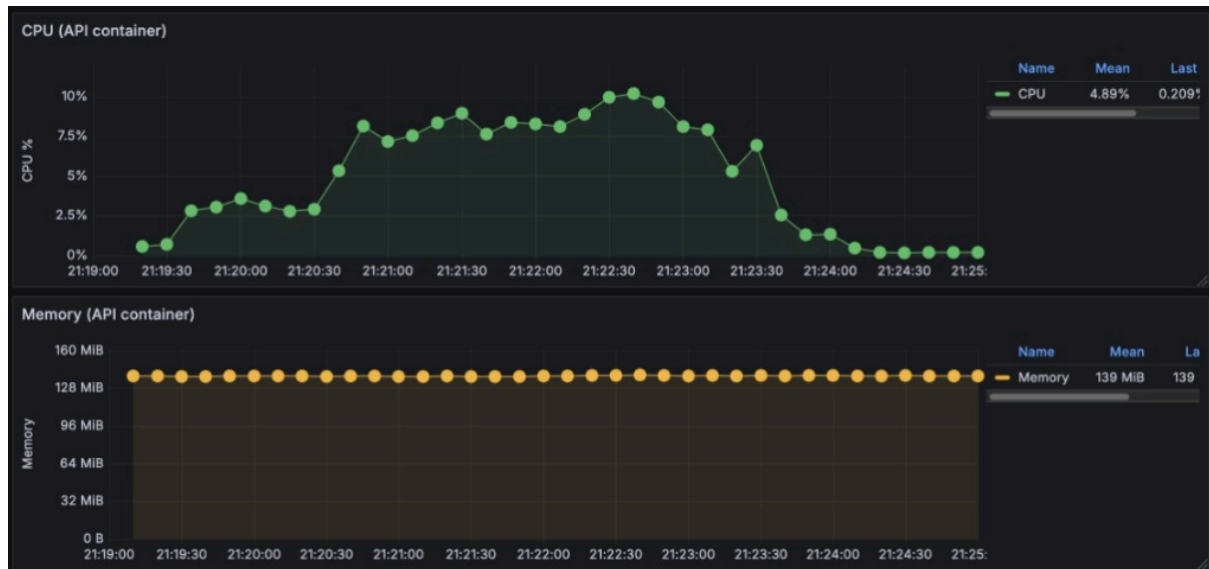
Tras unos primeros 20 segundos de “calentamiento” donde se procesan 5 req/s, los siguientes 60 segundos se eleva a 50 req/s el consumo de este endpoint. Posteriormente, los siguientes 120 segundos se da el pico de consumo mencionado previamente, simulando una situación provocada, por ejemplo, por el anuncio de una determinada política económica. Una vez finalizada dicha fase de volatilidad de divisas, simulamos que “se calma el mercado” y por lo tanto el uso vuelve a valores esperados de 50 req/s (durante sesenta segundos)



Una vez iniciada la fase donde simulamos la volatilidad de una divisa y por ende un uso creciente del endpoint en cuestión, el ritmo de solicitudes asciende desde 50 req/s hasta valores cercanos a 600 req/s. En este escenario no se observaron errores de cliente (vusers.failed = 0, ~61.900 respuestas HTTP 200). La mediana de latencia se mantuvo en el orden de milisegundos; el percentil alto (Upper) muestra un pico cercano a 1 s en torno al máximo de carga.

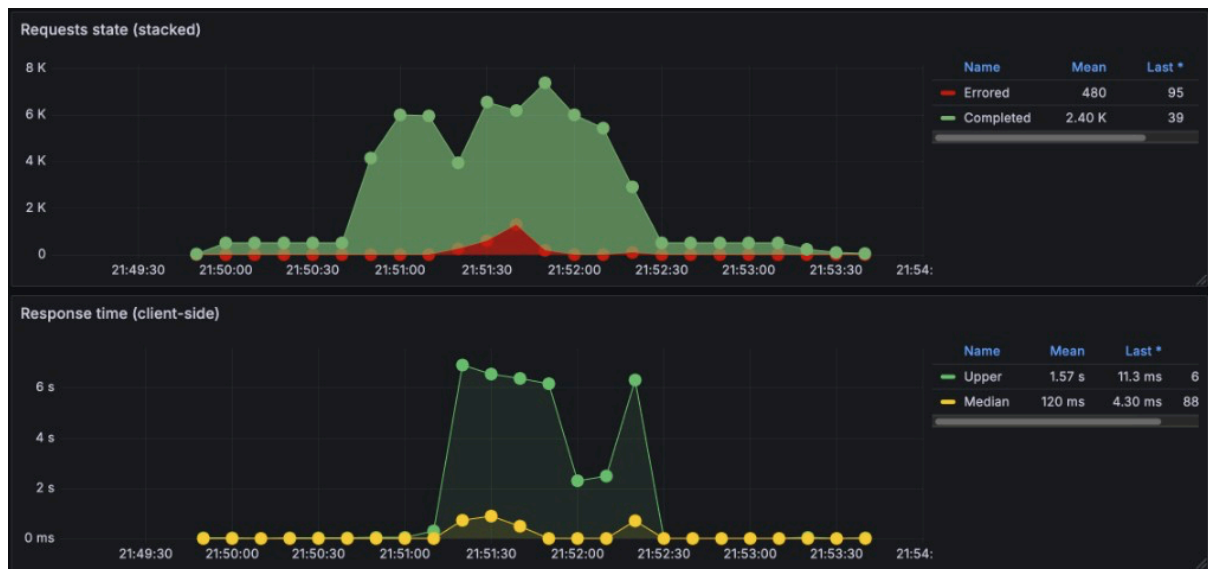
También se puede concluir que, a diferencia de lo que podría esperarse por una posible degradación progresiva fuerte en el tiempo de respuesta al aumentar la demanda, la mediana se mantiene en valores bajos y estables durante casi toda la corrida.

Además de las métricas de latencia y requests, se relevó el uso de recursos del contenedor de la API (exchange-api-1) durante la misma ejecución de la prueba gradual, a partir del panel Resources del dashboard de Grafana.



Se observa que la CPU acompaña las fases de mayor demanda, con picos cercanos al 10%, mientras que la memoria se mantiene estable (~139 MiB). En este escenario, entonces, no se asocia el comportamiento bajo alta req/s a un agotamiento de memoria ni a saturación de CPU del contenedor; más bien sugiere que el límite práctico está relacionado con la capacidad de atender conexiones y requests en el Event Loop.

Adicionalmente decidimos agregar un spike test para este endpoint donde el uso del mismo se presente de forma abrupta para observar qué errores se darían y si se vería perjudicada la performance, de forma tal que se observe una diferencia significativa con un uso gradual del servicio como observamos con la prueba anterior. En este caso el pool de conexiones fue establecido en 600 a diferencia del caso anterior donde se encontraba configurado en 200.

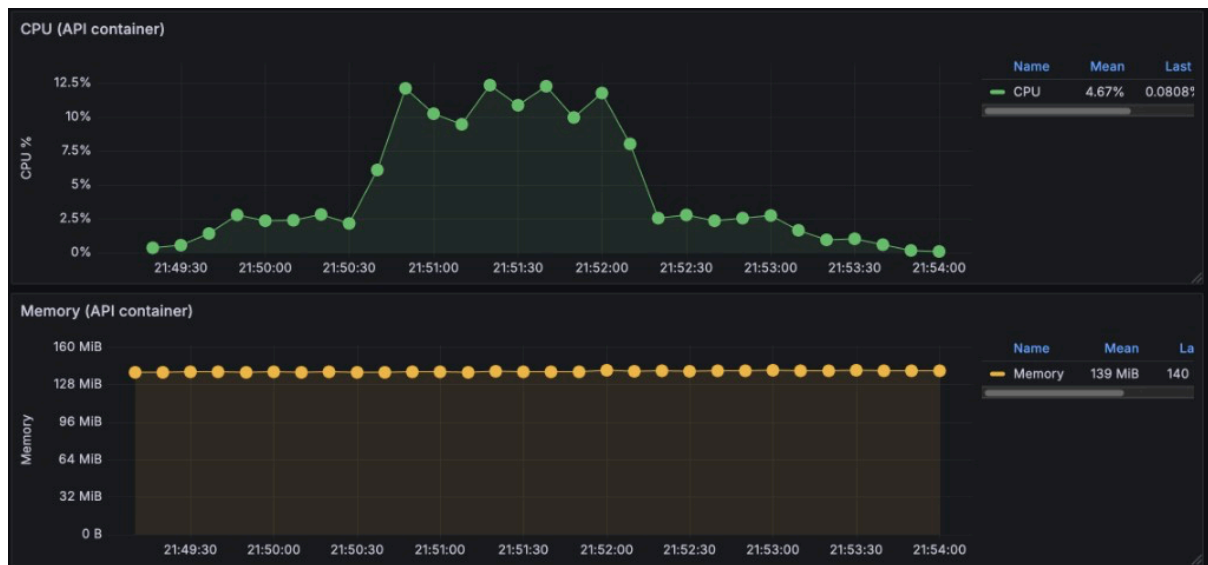


A simple vista se observa que claramente la performance se ve perjudicada cuando el servicio se encuentra bajo una demanda abrupta. Particularmente se observa que el tiempo de respuesta máximo (Upper) ronda los 7 s y que los errores comienzan a presentarse durante la fase de 600 req/s. Adicionalmente se registraron ETIMEDOUT y EAGAIN; al final de la prueba se recopilaron los siguientes datos:

```
errors.EAGAIN: 520
errors.ETIMEDOUT: 1879
http.codes.200: 58001
vusers.failed: 2399
```

También podemos observar que al comenzar la aparición de errores, se perjudica de forma crítica la disponibilidad (y por ende la fiabilidad) del servicio, ya que durante un momento de alta volatilidad, el servicio no es capaz de dar abasto (y por lo tanto en esta única prueba podemos observar como claramente se ven perjudicados 3 de los 4 atributos de calidad indicados como primarios para el servicio en cuestión).

En esa ejecución del escenario abrupto se relevó también el uso de recursos.

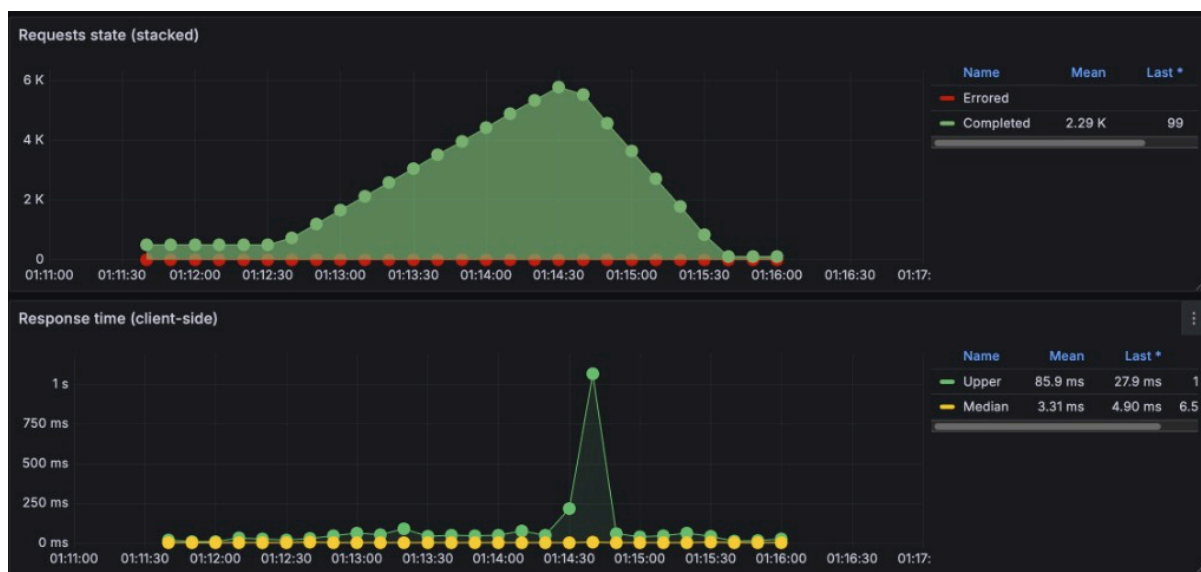


El patrón se mantiene: pico inicial de CPU cercano al 12–13% y luego valores bajos cuando ya predominan errores y latencias de varios segundos, con memoria estable (~139 MiB). Esto refuerza que el colapso bajo ráfaga no requiere saturar CPU/RAM del contenedor.

4.2.2. Evaluación de la actualización de balances sobre cuentas (PUT `/accounts/:id/balance`)

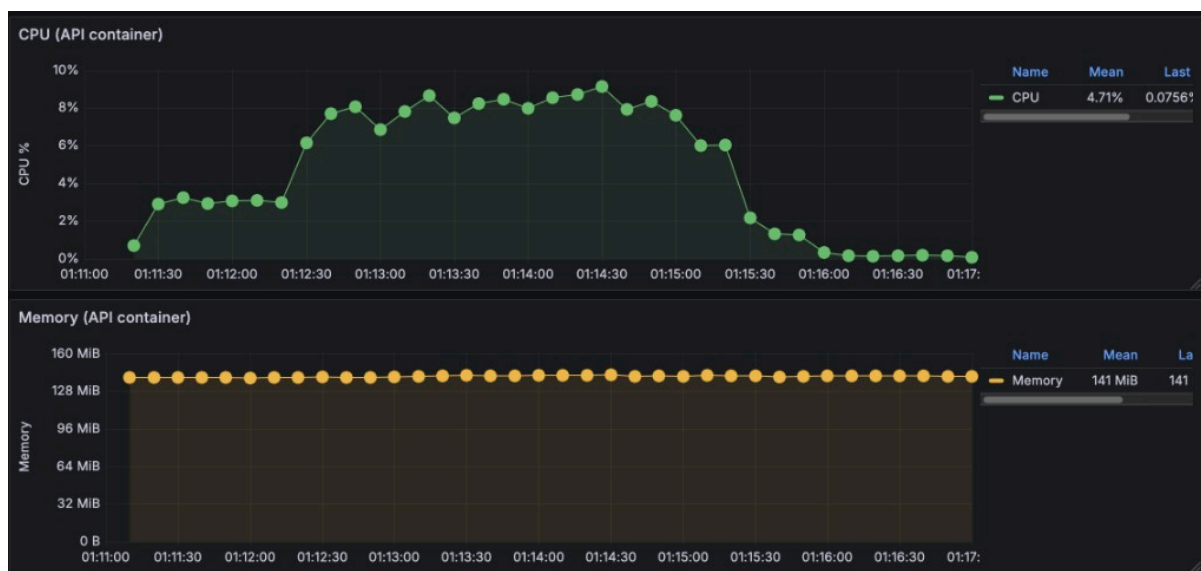
Si bien este endpoint se encuentra indicado en la documentación como posible responsabilidad de un servicio externo (como el encargado de realizar las transferencias) y por lo tanto podría ser una funcionalidad a eliminar para una versión de MVP, llegamos a la hipótesis de que el comportamiento a observar podría presentar similitudes con el caso del endpoint PUT `/rates` ya que el código simplemente valida la recepción de parámetros y posteriormente se actualiza el balance de la cuenta para la cual se realizó la solicitud.

Procedimos a realizar una prueba de carga donde se presenta un consumo que asciende gradualmente y luego vuelve a valores esperados.



Tras la fase de uso “normal” (50 req/s), comienzan a incrementarse las solicitudes durante 120 segundos hasta valores cercanos a 600 req/s, para luego recuperar. En esta corrida no se observaron errores de cliente (vusers.failed = 0, 61.900 respuestas HTTP 200). La mediana de latencia se mantiene en el orden de milisegundos durante casi toda la prueba; el percentil alto (Upper) muestra un pico cercano a 1 s en torno al máximo de carga, en línea con lo observado para PUT /rates bajo demanda gradual.

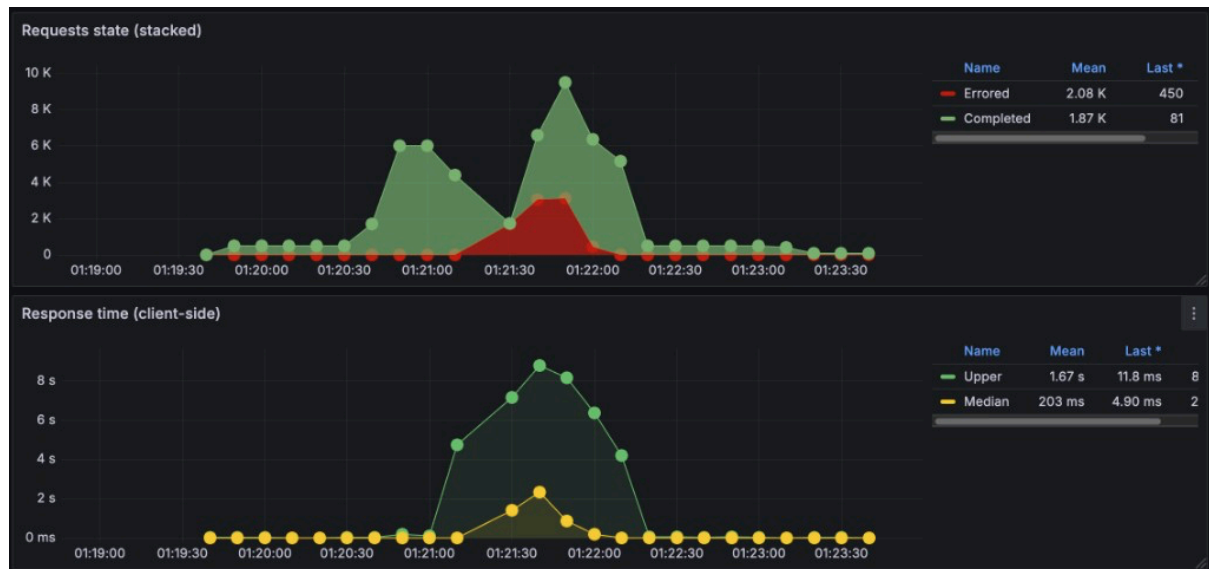
En cuanto al uso de recursos obtuvimos este resultado.



Nuevamente la CPU permanece baja (picos cercanos al 9%) y la memoria estable (~141 MiB), en línea con lo observado en /rates.

Al igual que para el caso de PUT /rates hemos realizado una prueba de carga abrupta (stress/spike). Entendemos que es de suma importancia indicar que si bien realizamos la prueba con la última cuenta en accounts (debido a la búsqueda lineal realizada en el código para aplicar la actualización del balance), de existir algún endpoint de creación de cuentas,

sería de suma relevancia incluir las métricas obtenidas sobre alguna prueba en distintas cuentas, por ejemplo la primera y la última de forma que si existiese una cantidad considerable de cuentas, sería interesante comparar los valores obtenidos al contemplar esta complejidad de búsqueda lineal sobre la cantidad de cuentas existentes (para este caso, al haber únicamente cuatro cuentas termina siendo despreciable).

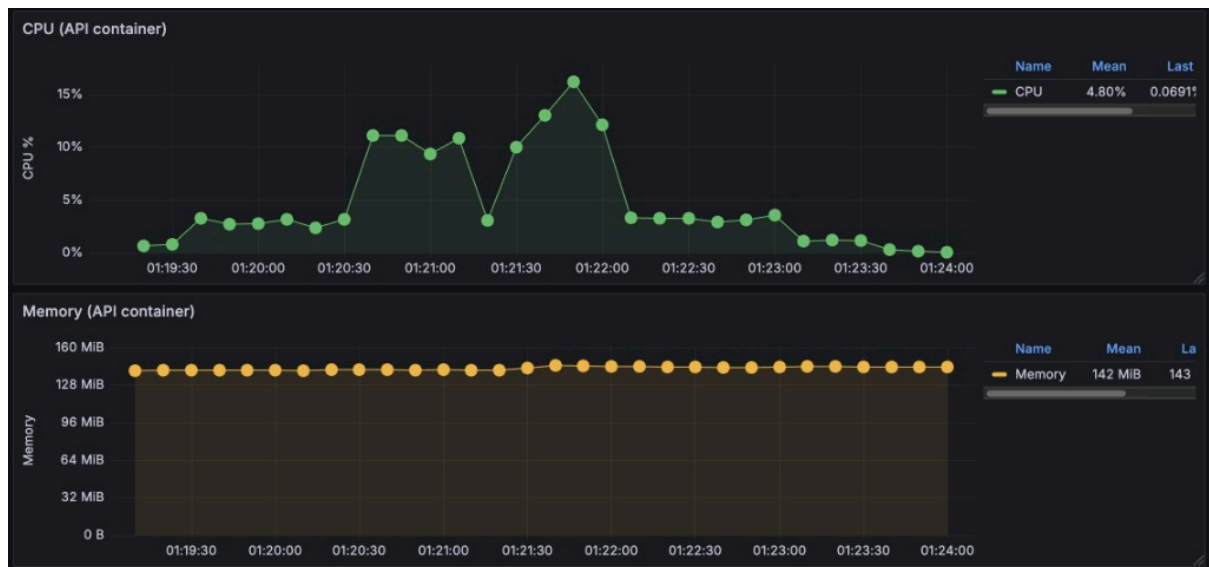


A simple vista se puede observar que en este strike test el tiempo de respuesta (Upper) supera los 8 s y que gran cantidad de solicitudes concluyen con error. Se observa que durante la fase donde se establecen unas 600 req/s comienzan a presentarse errores en las solicitudes (lo cual asciende de la mano con el tiempo de respuesta). Al final de la prueba se recopilieron los siguientes datos:

```
errors.EAGAIN: 730
errors.ETIMEDOUT: 8595
http.codes.200: 51075
vusers.failed: 9325
```

Sin embargo una vez normalizada (también de forma abrupta y no lineal o gradual como en el caso anterior) la cantidad de solicitudes realizadas, observamos que el servicio vuelve a responder de forma exitosa (tiempos de respuesta del orden de unos pocos milisegundos).

Por último, se relevó el uso de recursos del contenedor de la API en una ejecución del escenario abrupto.

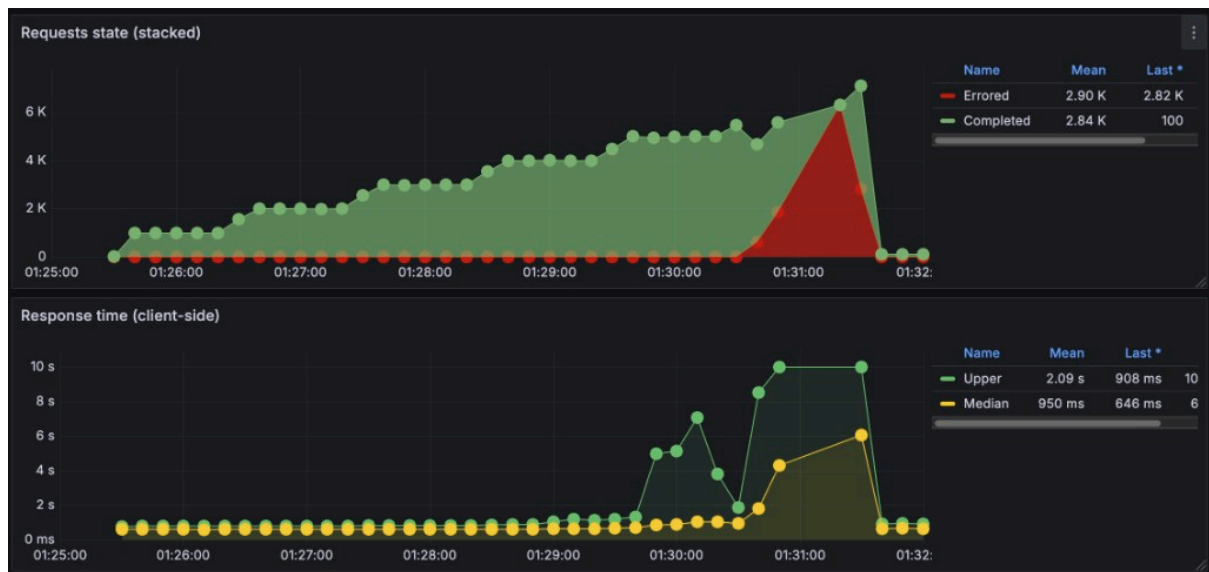


Pico de CPU cercano al 16–17% durante el salto; luego valores moderados/bajos con errores y latencias de varios segundos. Memoria estable (~142 MiB). Misma lectura: la falla aparece sin saturar los recursos del contenedor.

4.3. Medición sobre la operación de cambio de monedas

4.3.1. Evaluación del cambio de monedas (POST `/exchange`)

Para este caso el cual es posiblemente el de mayor relevancia dentro de los endpoints que dispone el servicio, llevamos adelante una prueba de carga progresiva en la cual durante 60 segundos aumentamos en cada fase la cantidad de req/s. Esta prueba cuenta con una fase de “calentamiento” que dura un total de 20 segundos para una cantidad de 5 req/s, para posteriormente continuar de forma escalonada con 6 fases de 60 segundos, incrementando desde las 100 req/s hasta las 600 req/s para finalmente concluir con una fase de “enfriamiento” donde la cantidad de req/s es disminuía a 10 req/s.



Particularmente se observa que el servicio deja de responder correctamente una vez alcanzadas las 500–600 req/s: la latencia Upper comienza a degradarse en torno a las 500 req/s y los errores aparecen de forma masiva al llegar a las 600 req/s. Previo a este instante, los tiempos de respuesta se ubican en el orden de cientos de milisegundos (recordemos que el tiempo de respuesta mínimo posible se ubicaría entre los 400 y 800 ms debido al timeout configurado en la simulación que realiza la transferencia); la mediana global de la corrida fue de 685.5 ms. Por otro lado, se observa que el tiempo de respuesta máximo registrado ronda los 9–10 s (en la fase de 600 req/s). Al final de la prueba se recopilaron los siguientes datos:

```
errors.ECONNRESET: 26
errors.ETIMEDOUT: 17974
http.codes.200: 108998
vusers.failed: 18000
```

Finalmente concluimos que el techo registrado para el cual aún el servicio se encontraba respondiendo de forma correcta fue el de las 500 req/s (pero en el mismo los tiempos de respuesta ya habían superado el segundo).

Complementariamente, se relevó el uso de recursos del contenedor de la API en una ejecución de este escenario de carga.



A diferencia de las escrituras livianas, la CPU es más sostenida y elevada (media cercana al 36%, picos cercanos al 75%), coherente con el costo de las dos transferencias simuladas por operación. La memoria, a diferencia de `/rates` y `/accounts`, crece de forma continua a lo largo de la corrida (~140 MiB → ~325 MiB). En degradación, entonces, al límite por Event Loop/conexiones se suma acá la latencia estructural del flujo de exchange y un uso de recursos del contenedor claramente mayor.

5. Tácticas aplicadas y Propuestas de mejora

5.1. Táctica 1: Control de Concurrencia en Memoria y Atomicidad en POST `/exchange`

- **Atributos de Calidad Impactados:** Fiabilidad (Reliability) e Integridad de Datos.
- **Diagnóstico del Problema en la Línea Base:** Durante las pruebas de carga sobre el endpoint POST `/exchange`, se identificó una vulnerabilidad crítica de integridad de datos derivada de la naturaleza asíncrona y monohilo (*single-threaded*) de Node.js.

En la implementación original, la función `exchange()` cedía el control del Event Loop mediante la sentencia `await transfer(...)` en dos momentos sucesivos de su ejecución. Bajo peticiones concurrentes sobre un mismo par de cuentas:

1. La Petición A leía el saldo de la cuenta del cliente (ej. USD 100) y cedía la ejecución al esperar la transferencia del primer tramo.
2. Antes de que la Petición A completara el débito, la Petición B leía el **mismo saldo desactualizado** (USD 100).

3. Ambas peticiones evaluaban fondos suficientes y procedían a debitar, generando un doble gasto, saldos negativos e inconsistencias financieras irreversibles.
- **Descripción e Implementación de la Táctica:** Para erradicar las condiciones de carrera sin bloquear el Event Loop con operaciones sincrónicas pesadas, se implementó un mecanismo ligero de **Mutex en memoria por par de cuentas** (`acquireLocks`) en la capa de aplicación (`app/exchange.js`).

Las decisiones clave de diseño fueron:

- **Cola de Promesas por Clave de Cuenta:** Cada cuenta posee su propia cola de resolución en el objeto global `accountLocks`. Cuando una transacción involucra a una cuenta, las transacciones subsiguientes sobre la misma cuenta esperan asincrónicamente a que finalice la anterior.
- **Ordenamiento Determinista para Prevención de Deadlocks:** Para evitar interbloqueos (deadlocks) entre transferencias recíprocas (ej. Cuenta A → Cuenta B y Cuenta B → Cuenta A simultáneamente), los IDs de las cuentas involucradas se ordenan alfabéticamente antes de solicitar el acceso. De esta forma, cualquier transacción compite por los recursos siempre en el mismo orden rígido.
- **Garantía de Liberación mediante `try...finally`:** La ejecución de la transferencia y actualización de estado se envuelve en un bloque de control estricto que asegura la liberación incondicional de las claves asignadas, impidiendo bloqueos permanentes ante fallos de validación o errores no contemplados.

Trade-offs y Consideraciones Arquitectónicas:

- **Ventajas:** Resuelve de forma inmediata e incondicional la corrupción de saldos sin introducir latencia de red ni dependencias de infraestructura externa (como bases de datos relacionales o caches distribuidos).
- **Límite de la Táctica:** Esta solución garantiza atomicidad únicamente mientras la API se ejecute como una instancia monolítica en un solo proceso Node.js. Al evolucionar hacia una arquitectura distribuida con múltiples réplicas detrás de un balanceador de carga, el mutex local servirá como base para ser migrado a un mecanismo de Lock Distribuido o Transacciones Atómicas en Redis (`MULTI/EXEC` o Scripts Lua).

Verificación y Resultados Obtenidos:

Para evidenciar la condición de carrera se ejecutó el mismo escenario Artillery en dos variantes de la arquitectura. El escenario fija primero el saldo de la cuenta USD en 2 y luego aplica presión concurrente sobre POST `/exchange`.

Sin mutex, tras la corrida GET `/accounts` mostró saldo USD negativo (-10,54): varios requests concurrentes evaluaron el check de fondos sobre el mismo valor previo y debitaron en paralelo.

```
[
  { "id": 1, "currency": "ARS", "balance": 120019000 },
  { "id": 2, "currency": "USD", "balance": -10.540000000000001 },
  { "id": 3, "currency": "EUR", "balance": 40000 },
  { "id": 4, "currency": "BRL", "balance": 60000 }
]
```

Con mutex, bajo el mismo escenario, el saldo USD final fue $\approx 0,02$ (no negativo); sólo se aplicaron las operaciones serializadas que aún tenían fondos suficientes (pocas respuestas HTTP 200) y el resto fue rechazado por fondos insuficientes (HTTP 500).

```
[
  { "id": 1, "currency": "ARS", "balance": 120003000 },
  { "id": 2, "currency": "USD", "balance": 0.019999999999999796 },
  { "id": 3, "currency": "EUR", "balance": 40000 },
  { "id": 4, "currency": "BRL", "balance": 60000 }
]
```

5.2. Táctica 2: Protección Perimetral contra Saturación y Control de Tasa (Rate Limiting en Nginx)

- **Atributos de Calidad Impactados:** Disponibilidad (Availability) y Robustez / Tolerancia a Fallos (Graceful Degradation).
- **Diagnóstico del Problema en la Línea Base:** Durante las pruebas de carga baseline (observadas en GET `/rates`, GET `/accounts` y POST `/exchange`), se constató que al incrementar la tasa de tráfico por encima de las 300 req/s y aproximarse a las 500 req/s, el contenedor de la API alcanzaba el 100% de saturación de CPU.

Al ser Node.js monohilo, el Event Loop se vio completamente desbordado procesando el handshake de red, el parseo de encabezados y la serialización JSON de peticiones masivas. Como consecuencia, las conexiones entrantes quedaron encoladas indefinidamente en el buffer TCP del sistema operativo hasta que el cliente (Artillery) abortó las solicitudes por expiración de tiempo de espera (`ETIMEDOUT`), provocando un colapso total del servicio y la indisponibilidad de la plataforma para cualquier usuario legítimo.

- **Descripción e Implementación de la Táctica:** Para mitigar el colapso por saturación y evitar que el tráfico excesivo degrade el backend, se implementó la táctica canónica de **Control de la Tasa de Solicitudes (Manage Work Rate / Throttling)** en la capa perimetral del Proxy Inverso (Nginx en `nginx_reverse_proxy.conf`).

Las decisiones clave de configuración fueron:

- **Definición de Zona de Memoria Compartida (`limit_req_zone`):** Se configuró una zona en memoria de 10 MB (`zone=api_rate_limit:10m`) indexada por la IP binaria del cliente (`$binary_remote_addr`, ocupando solo 4 bytes por cliente IPv4). Se estableció una tasa máxima operativa de procesamiento de **300 peticiones por segundo** (`rate=300r/s`), correspondiente al límite de estabilidad medido empíricamente en la API.
- **Tolerancia a Micro-Ráfagas sin Retardo (`burst=50 nodelay`):** Se incorporó un buffer de 50 peticiones en ráfaga con la directiva `nodelay`. Esto permite admitir picos legítimos de corta duración procesándolos de inmediato a velocidad nominal, sin introducir retardos artificiales de encolamiento.
- **Fallo Rápido y Respuesta Explícita (`limit_req_status 429`):** Se sobrescribió el código de respuesta por defecto de Nginx (503) para retornar HTTP 429 Too Many Requests. De este modo, las solicitudes que superan el umbral son interceptadas y rechazadas en microsegundos en el perímetro, sin consumir ciclos de CPU ni memoria del servicio [Node.js](#).

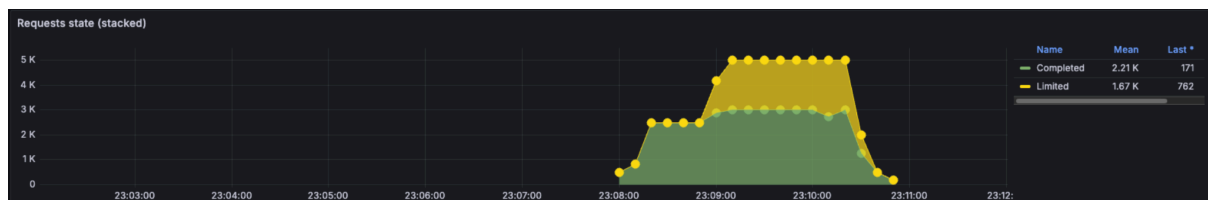
Trade-offs y Consideraciones Arquitectónicas:

- **Ventajas:** Aislamiento total del Event Loop de la API frente a sobrecargas o ataques de denegación de servicio (DoS). El sistema mantiene un rendimiento predecible y tiempos de respuesta estables para el tráfico admitido, garantizando que el servicio nunca muera por `ETIMEDOUT`.
- **Consideraciones:** Se adopta una estrategia de degradación controlada (load shedding): las peticiones que exceden la capacidad contratada son rechazadas activamente en lugar de ser encoladas indefinidamente. Para

clientes legítimos con alta demanda, la arquitectura requerirá complementar esta táctica con escalamiento horizontal en la capa de cómputo..

Verificación y Resultados Obtenidos:

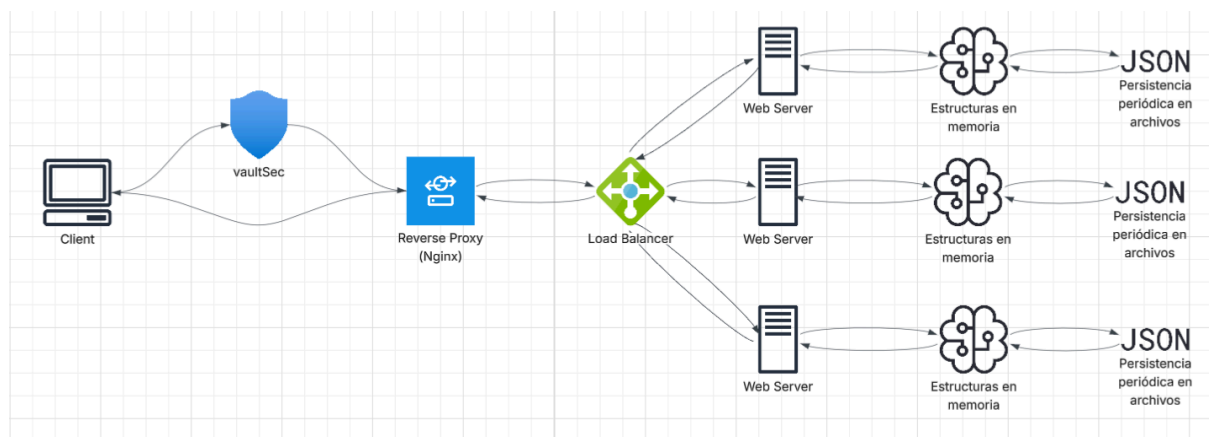
Se ejecutó un escenario Artillery sobre GET `/rates` con una fase bajo el umbral (~250 req/s) y otra por encima (~500 req/s), sobre la rama con rate limiting en Nginx (`limit_req` a 300 r/s). En la corrida completa se obtuvieron 40.433 respuestas HTTP 200 y 18.317 respuestas HTTP 429, sin errores `ETIMEDOUT`. Durante la fase a 500 req/s, el ritmo de respuestas exitosas se estabiliza en torno a ~300 req/s y el excedente es rechazado con 429, coherente con el límite configurado. Esto valida la táctica de degradación controlada en el perímetro: el tráfico admisible se atiende y el excedente se descarta de forma explícita, evitando el colapso por timeouts observado en el caso base.



5.3. Táctica 3: “Scale out not up”, “Stateless” y “Redistribute Responsibilities”

- **Atributos de Calidad Impactados:** Disponibilidad , Fiabilidad y Performance.
- **Diagnóstico del Problema en la Línea Base:** Durante el análisis del código inicial se indicó que el servidor web además de encargarse de la lógica del servicio también mantiene y persiste los datos (en memoria y con volcado a disco periódico). Si queremos sumar instancias del servidor web, cada una contaría con su propia información de la cual los demás no serían capaces de enterarse tras ser actualizada, de forma que se pueden dar transferencias inválidas según los balances de las cuentas.

Como indicamos inicialmente, la persistencia de rates, accounts y log se realiza en memoria con un volcado periódico a archivos JSON, sin una base de datos. Dado que disponibilidad y performance son atributos principales de este servicio, al plantear la replicación del web server (varias instancias de Node.js detrás de un balanceador) identificamos como principal obstáculo el estado local: cada proceso mantiene sus propias estructuras en memoria y su propia copia de los archivos, por lo que las instancias divergen entre sí. Esto compromete la consistencia de los datos, y con ella la fiabilidad, además de perder información ante una caída entre volcados.

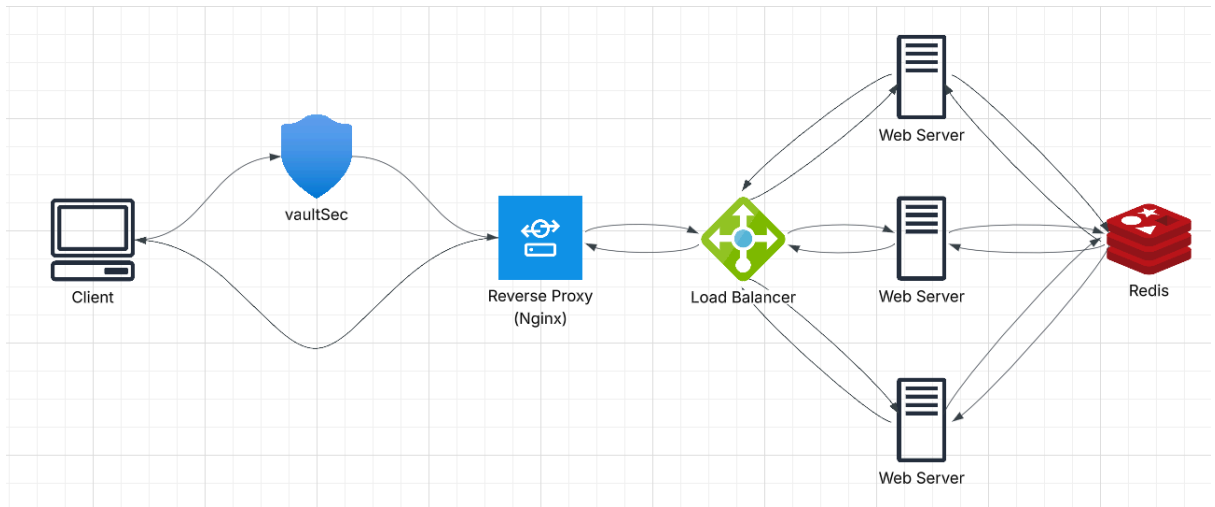


Para resolverlo, externalizamos el estado a Redis, un almacenamiento compartido en el cual se delega el mantenimiento del estado. Como las operaciones concurrentes de distintas instancias podían gastar el mismo saldo, la reserva de fondos se implementó como un script Lua (el cual se ejecuta de forma atómica), mientras que para aquellos casos donde los endpoints cumplían roles de menor complejidad como “getters” o “setters” simplemente se hace uso de las transacciones tradicionales de Redis (`multi()` -> con encadenamiento de `get()/set()` -> `exec()`) lo cual también provee una mejora de performance en caso de que tanto rates como accounts crezcan de cara al futuro debido a que la complejidad de `get()/set()` es $O(1)$ lo cual es una mejora respecto de los $O(N)$ indicados en el análisis del código inicial. Además, Redis cuenta con persistencia AOF y por ende delegamos en él ese volcado a disco periódico que realizamos anteriormente (el **fsync** es por segundo, no confundir con extinguir la posibilidad de perder datos) y nos provee de un mecanismo de recuperación.

Por lo tanto hemos aplicado las siguientes tácticas:

- **Maintain multiple copies of computations / Active redundancy / Scale out:** tres instancias del servicio y nginx como balanceador con round robin, con chequeos de salud pasivos para sacar de rotación instancias caídas.
- **Stateless:** Hacemos uso de Redis como almacenamiento/caché distribuido entre nuestras instancias del servicio, separando la persistencia de las unidades lógicas del servicio.

- **Redistribute responsibilities:** Ahora con Redis dentro de nuestro servicio, al mismo se delega la responsabilidad de la persistencia y acceso a la información.



Sin embargo es de suma importancia señalar que si bien anteriormente la única instancia de nuestro servidor web era un único punto de falla (y también nginx debido a que era el único “lugar” al cual llegaban las solicitudes y estas eran “reenviadas”), tras estos cambios debemos reconocer a Redis como nuevo punto único de falla. Adicionalmente, el uso de Redis provocó una modificación en cuanto a la transaccionalidad aplicada mediante la primera táctica dado que debimos implementar el uso de un script Lua que reemplaza la implementación realizada en `exchange()`. Para evitar que instancias concurrentes gasten el mismo saldo, la reserva de fondos que se implementó como un script Lua ejecutado en Redis verifica el saldo y lo descuenta en una única operación atómica, evitando condiciones de carrera entre instancias. El resto de la operación de cambio no es transaccional en sentido estricto: se resuelve mediante compensaciones (si falla alguna transferencia, se devuelven los fondos reservados y se revierte lo transferido al cliente). Esto garantiza consistencia en el caso habitual y ante fallos controlados, pero no ante la caída de una instancia en medio de una operación, lo cual significa una limitación.

En una próxima iteración podría replicarse nginx (con IP virtual mediante keepalived donde contemos con una instancia secundaria de nginx capaz de reemplazar a la principal) y Redis (con réplicas y Sentinel para realizar monitoreo y reemplazo en caso de que la instancia primaria vea interrumpido su funcionamiento), teniendo en cuenta que la replicación de Redis es asincrónica y puede perder las últimas escrituras en un failover, también se puede aplicar sharding con Redis cluster para repartir los datos en distintos nodos.

Las decisiones clave de configuración fueron:

- **Balanceo de carga entre réplicas del servidor web**

(`nginx_reverse_proxy.conf`):

- Se define un **upstream** (`exchange_api`) con las tres instancias del servicio (`api1`, `api2`, `api3`, puerto 3000) y nginx actúa como reverse proxy: es el único punto de entrada de los clientes. Se utiliza el algoritmo por defecto, round robin.

- Detección de fallas. Cada servidor se declara con `max_fails=3` y `fail_timeout=10s`, lo que implementa un chequeo de salud pasivo: si una instancia falla 3 veces, nginx la saca de rotación durante 10 segundos y luego vuelve a probarla. Esto permite que el sistema siga respondiendo ante la caída de una instancia sin intervención manual.
 - Reintentos. Con `proxy_next_upstream error timeout`, ante un error de conexión o un timeout la solicitud se reenvía a otra instancia. No se habilitó `non_idempotent`, por lo que los `POST /exchange` no se reintentan: esto evita ejecutar dos veces un cambio de moneda si una instancia falla a mitad de la operación. Se prioriza la integridad de las operaciones sobre la disponibilidad de ese endpoint puntual.
 - Observabilidad. Se agrega el header `X-Served-By ($upstream_addr)` para verificar qué instancia atendió cada solicitud y comprobar el reparto de carga. Esto simplemente contribuye al desarrollo y su validación.
- **Implementación de Redis (docker-compose.yml):**
 - Tres instancias del web server. `api1`, `api2` y `api3` se construyen desde la misma imagen (`build: ./app`) y comparten configuración mediante un YAML anchor (`&api / <<: *api`), de modo que las tres tienen los mismos límites de recursos (`cpus: "1.0"`, `mem_limit: 512m`). Esto garantiza homogeneidad, condición necesaria para que round robin reparta la carga de forma equitativa, y evita duplicar configuración. Solo difiere `INSTANCE_ID`, que identifica cada réplica. Las instancias no publican puertos al host: solo son accesibles a través de nginx, que expone el puerto 5555.
 - Redis como estado compartido. Se incorpora un servicio `redis` (`redis:7-alpine`) al que las tres instancias acceden mediante `REDIS_URL=redis://redis:6379`, resuelto por DNS interno de Docker Compose. De esta forma las instancias quedan sin estado. Se inicia con `--appendonly yes` (persistencia AOF, con `fsync` por segundo por defecto) y un volumen nombrado `redis_data`, de modo que los datos sobreviven al reinicio o recreación del contenedor. Esto reduce la ventana de pérdida frente al volcado periódico anterior, pero no la elimina.
 - Orden de arranque. `nginx` depende de las tres instancias y estas de `redis`. Cabe aclarar que `depends_on` solo espera a que el contenedor haya *iniciado*, no a que el servicio esté listo. Las instancias toleran que Redis tarde unos instantes porque el cliente `ioredis` reintenta la conexión, y la siembra inicial está coordinada con un lock

(`seed:lock` / `seed:done`) para que solo una instancia cargue los datos.

- Limitaciones reconocidas. `nginx` y `redis` son instancias únicas y, por lo tanto, puntos únicos de falla. Además, las tres réplicas corren en un mismo host, así que protegen ante la falla de un proceso o contenedor, pero no ante la caída de la máquina.

Trade-offs sobre atributos de calidad:

- **Ventajas:** Mejoramos disponibilidad (ante fallas de instancia), performance en throughput dado que dividimos el cómputo entre ellas, fiabilidad (consistencia y durabilidad) por el uso de Redis y escalabilidad (horizontal).
- **Desventajas:** Disminuye la simplicidad, aumenta el costo de infraestructura (modificabilidad), latencia individual por request (por los saltos extra de red) y testabilidad.

5.4. **Táctica 4:** “Disponibilidad” y “No repudio” desde el punto de vista de la Seguridad

La disponibilidad de la aplicación es clave por lo que prevenir un ataque DoS puede ser vital para el correcto funcionamiento de la misma, además en un mercado competitivo como el financiero estar cubiertos a ataques informáticos no es un detalle menor.

Detectamos que la versión inicial no cubría un ataque DoS por lo que implementamos un conjunto de reglas en el servidor de NGINX que permitirán estar al menos cubiertos ante algunos de los casos.

Apoyándonos en el uso de un rate limit como se menciona en la 2da táctica le sumamos al mismo connection timeouts, límite en el tamaño del body enviado por el cliente y máximo de conexiones simultáneas por IP.

Por supuesto que esto no cubre todos los casos ni cubre el ataque DDoS que es el más crítico pero son pequeñas modificaciones que sin afectar la arquitectura actual aumentan la calidad del QA Seguridad.

Otra modificación que hicimos en pos de mejorar el QA fue explicitar los contratos que la app presenta, la versión inicial deja mucho que desear en términos de contratos en lo que refiere al exchange como se mencionó en el apartado de seguridad. Uno de los más importantes es el tema de la tarifa, un cliente A confirma un exchange de la moneda B al precio C, por como es el diseño inicial ese precio podría haber cambiado en el medio a C+D lo cual implicaría un contrato corrupto. Para mitigar este problema lo que se hizo fue recibir como parámetro en el body la tarifa de referencia y agregar un chequeo de equidad entre la tarifa recibida y la utilizada al momento de la transacción. Este pequeño cambio no tiene costo a nivel performance ya que es una operación del tipo $O(1)$ en términos de complejidad temporal y el dato recibido tiene un peso de 8 bytes. Se modificó el contrato del endpoint `POST /exchange`, exigiendo `expectedRate` en el body de la request y agregando el código HTTP `409 Conflict (RATE_CHANGED)` alegando que ese rate no se encuentra más disponible.

Estas tácticas pueden integrarse prácticamente sin costo con las demás propuestas.

- **Ventajas:** Mejoramos la seguridad e integridad de la información. La disponibilidad también se ve beneficiada.
- **Desventajas:** Disminuye la simplicidad aunque de forma mínima.

6. Combinando tácticas

Se combinaron las tácticas 2, 3 y 4 ya que la 1 dejaba de ser útil una vez introducida la arquitectura distribuida, además de que **reserveFounds** implementado en la 2da táctica tiene el mismo objetivo.

La prueba buscó revisar dos cosas:

1. Cómo responde NGINX cuando la carga se acerca y supera las 3000 requests por segundo.
2. Cómo se comporta el endpoint `POST /exchange` con tres réplicas, Redis y una variedad mayor de clientes.

Entorno utilizado

- Tres instancias de la API detrás de NGINX.
- Una instancia de Redis compartida.
- Artillery para generar tráfico.
- Graphite y Grafana para visualizar los resultados.
- Diez contenedores de Artillery en la prueba de exchanges.
- Una computadora local con Colima. Los resultados sirven para comparar el comportamiento de esta versión, pero no representan la capacidad de un ambiente de producción.

Prueba del límite de requests

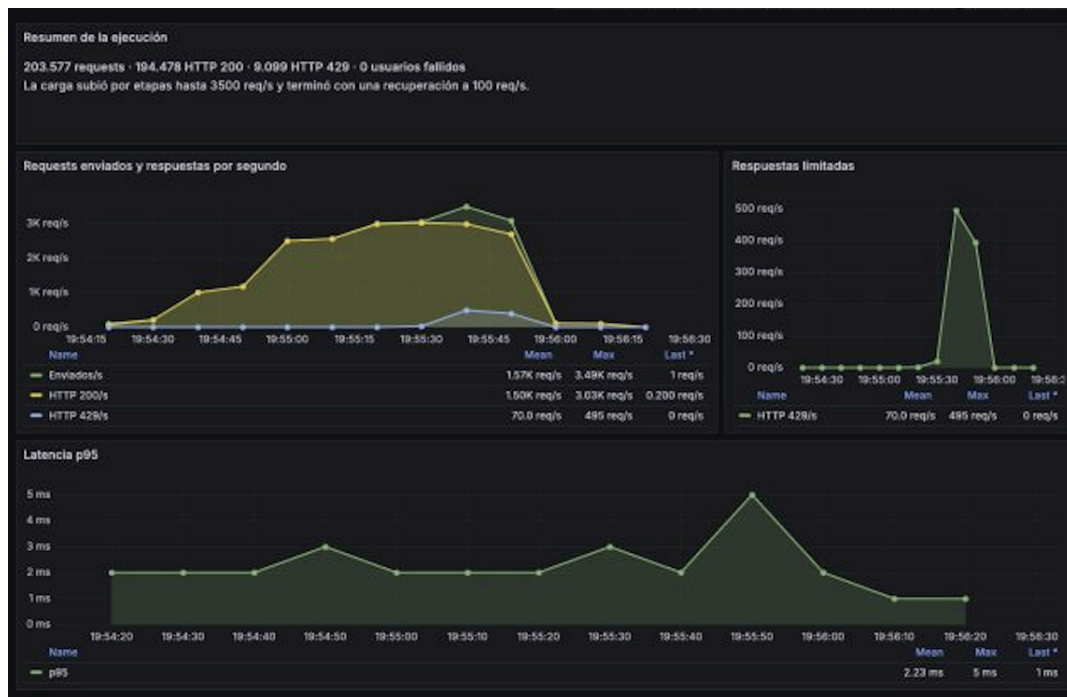
El límite de NGINX se configuró en 3000 requests por segundo, con un `burst` de 600. Se usó `GET /rates` porque permite generar carga sobre NGINX y las réplicas sin mezclar el resultado con los 400 a 800 ms que demora un `exchange` por las dos transferencias simuladas.

La carga se dividió de la siguiente forma:

Etapas	Duración	Carga enviada
Entrada	15 s	100 req/s
Carga media	20 s	1000 req/s
Carga alta	20 s	2500 req/s
Límite configurado	20 s	3000 req/s
Por encima del límite	20 s	3500 req/s
Recuperación	20 s	100 req/s

En total se enviaron 203.577 requests. Se recibieron 194.478 respuestas 200 y 9.099 respuestas 429. A 3000 req/s pasaron casi todas las requests. Al subir a 3500 req/s aparecieron los rechazos y la cantidad de respuestas exitosas se mantuvo cerca del límite configurado.

Los logs de NGINX permiten separar los motivos de los 429: 8.938 fueron producidos por `limit_req` y 161 por `limit_conn`. Cuando la carga volvió a 100 req/s no hubo nuevos rechazos y la latencia p95 volvió a 1 ms. Esto muestra que el servicio siguió respondiendo y se recuperó apenas bajó el tráfico.



Resultado de la prueba de 3000 requests por segundo

Prueba de exchanges

La API no guarda las cuentas externas de los clientes: la función transfer es una simulación y acepta los IDs recibidos. Por ese motivo se preparó un conjunto de 400 pares de IDs de clientes y cada request eligió uno al azar. Los IDs varían las solicitudes como lo harían distintos clientes, mientras que los fondos reales de la aplicación siguen siendo las cuatro cuentas internas compartidas en Redis.

Se utilizaron diez generadores para evitar que todo el tráfico saliera desde una sola IP y quedara limitado a 20 conexiones simultáneas. Cada generador siguió estas etapas:

Etapas	Duración	Carga por generador	Carga total
Entrada	20 s	1 exchange/s	10 exchange/s
Normal	45 s	5 exchange/s	50 exchange/s
Alta	60 s	10 exchange/s	100 exchange/s
Pico	60 s	20 exchange/s	200 exchange/s
Recuperación	30 s	2 exchange/s	20 exchange/s

El resultado fue el siguiente:

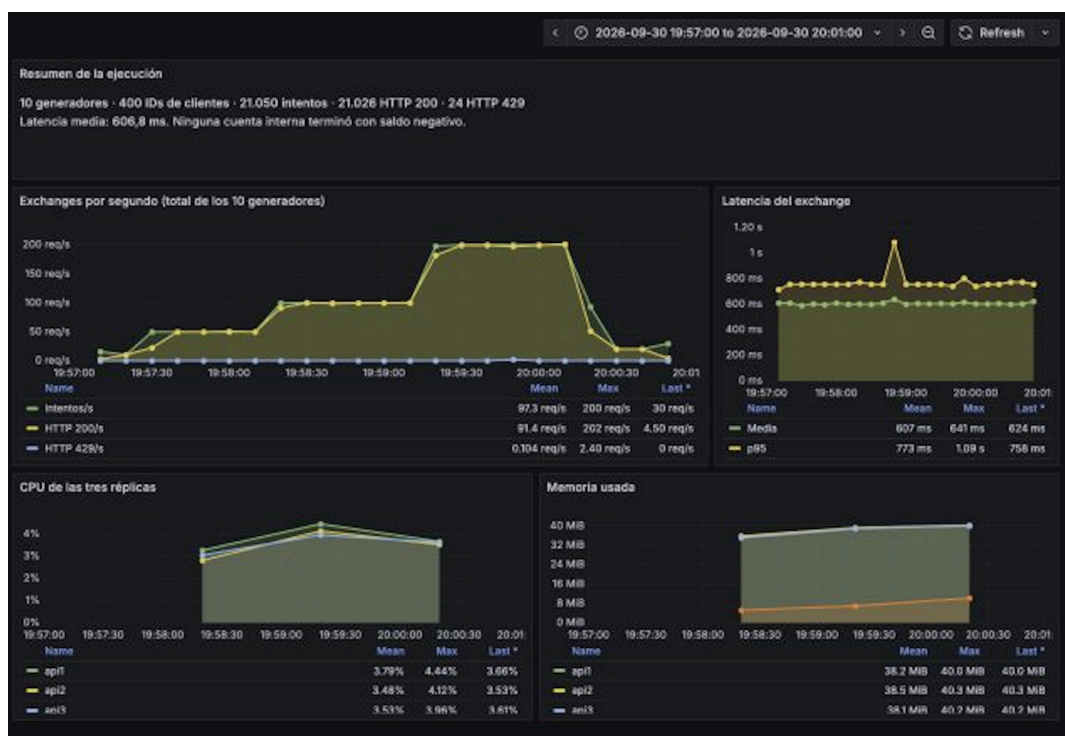
Resultado	Valor
Intentos	21.050
HTTP 200	21.026
HTTP 429	24
Porcentaje HTTP 200	99,886%
Latencia media	606,8 ms

Resultado	Valor
p95 por generador	entre 742,6 y 757,6 ms
p99 por generador	entre 788,5 y 804,5 ms
Máximo observado	1545 ms

Los 24 rechazos ocurrieron durante el pico y los logs los atribuyen a `limit_conn`. No hubo rechazos por el límite de 3000 req/s, algo esperable porque el máximo de esta prueba fue de 200 exchanges por segundo. La demora se mantuvo cerca de 600 ms durante todas las etapas. Este valor coincide con las dos transferencias simuladas, cada una con una espera aleatoria de 200 a 400 ms.

Las tres réplicas tuvieron un uso parecido. En las tres muestras guardadas estuvieron entre 2,8% y 4,44% de CPU y entre 35,1 y 40,3 MiB de memoria. Redis pasó de 5,2 a 10,3 MiB. No se observó una réplica concentrando la carga.

Al finalizar se revisaron las cuatro cuentas internas de Redis y ninguna tenía saldo negativo. El log contenía 21.026 exchanges, la misma cantidad que las respuestas 200. Los 24 pedidos limitados por NGINX no llegaron a la aplicación.



Resultado de la prueba de exchanges

Seguridad

Además de la carga se repitieron controles puntuales:

- Se abrieron 25 requests incompletas al mismo tiempo. NGINX mantuvo 20 conexiones y rechazó 5 con 429, que es el límite configurado por IP.
- Se cambió la tarifa de ARS a USD y luego se intentó confirmar un exchange con la tarifa anterior. La API respondió 409 con `RATE_CHANGED` y los saldos quedaron iguales.
- Un exchange sin `expectedRate` respondió 400.
- Un body de aproximadamente 20 KB respondió 413 porque supera el máximo de 16 KB.

```
qa-perf — Evidencia límite de conexiones — zsh — 100x34

COMANDO EJECUTADO:
python3 verify_connection_limit.py

SALIDA REAL:
{
  "executedAt": "2026-09-30T23:33:33.967782+00:00",
  "target": "http://127.0.0.1:5555/exchange",
  "attemptedConnections": 25,
  "openedSockets": 25,
  "pendingIncompleteRequests": 20,
  "rejectedWith429": 5,
  "connectionErrors": [],
  "immediateResponses": [
    "HTTP/1.1 429 Too Many Requests",
    "HTTP/1.1 429 Too Many Requests",
    "HTTP/1.1 429 Too Many Requests",
    "HTTP/1.1 429 Too Many Requests",
    "HTTP/1.1 429 Too Many Requests"
  ],
  "expected": {
    "pendingIncompleteRequests": 20,
    "rejectedWith429": 5
  },
  "passed": true
}
```

Ejecución real de la prueba de conexiones

```
qa-perf — Evidencia contratos HTTP — -zsh — 100x24

ARCHIVO REAL GENERADO POR LAS PRUEBAS:
docs/evidence/raw/contract-and-body-limit.json

RESULTADOS:
{
  "staleRateStatus": 409,
  "staleRateError": "RATE_CHANGED",
  "balancesUnchanged": true,
  "missingExpectedRateStatus": 400,
  "oversizedBodyStatus": 413
}
```

Resultados reales de los controles de contrato y tamaño del body

Conclusiones

El chequeo de `expectedRate` no produjo un costo visible en la prueba. Es una comparación hecha antes de reservar fondos y evita comenzar una operación cuando la tarifa ya cambió.

El rate limit tampoco intervino durante la prueba de exchanges porque la carga quedó muy por debajo de 3000 req/s. Su efecto se vio en la prueba específica de `GET /rates`, cuando se superó ese valor.

El límite de conexiones sí tuvo efecto sobre los exchanges. Como cada operación mantiene la conexión abierta unos 600 ms, un generador que llega a 20 operaciones por segundo puede superar por momentos las 20 conexiones simultáneas. Por eso se observaron 24 respuestas 429 en el pico.

La reserva atómica de Redis mantuvo los saldos internos válidos usando las tres réplicas. Esta prueba no cubre una caída de Redis ni una ejecución con varias instancias de Redis, ya que se utilizó un único servidor.

Utilizar redis no sumó latencia a pesar de estar distribuido en vez de un array en memoria, performando de manera excelente con un p99 de 6 ms.

7. Uso de IA

Se han utilizado herramientas de IA (asistentes basados en LLM) para los siguientes fines:

1. Generación y ajuste de scripts de prueba:
 - a. Artillery para las distintas pruebas realizadas (no así su análisis plasmado en el informe).
 - b. LUA para la reserva de fondos (utilizado en `exchange()`).
 - c. Prueba de DoS abriendo más de 20 conexiones.
2. Documentación y versionado:
 - a. Asistencia en la redacción para los títulos y descripciones de los Pull Requests en GitHub.

8. Conclusiones Finales

Luego de hacer un profundo análisis sobre la arquitectura de la aplicación determinamos que los atributos de calidad afectados de forma más crítica son Escalabilidad, Fiabilidad, Performance, Disponibilidad y Seguridad. Diversos motivos contribuyen a que se encuentren en un estado crítico.

La estructura inicial del servicio no favorece la disponibilidad y performance dado que emplea malas prácticas como devolver la lista de cuentas completa lo que satura el CPU con un volumen alto de cuentas pero asumimos que es un requerimiento funcional, por lo que implementar estrategias de rate limit, escalabilidad horizontal en conjunto con un load balancer y Redis para la persistencia ayudan a mitigar el problema. Del lado de la Fiabilidad y Seguridad se encontraron hallazgos preocupantes, debido a que las transacciones solo tienen el nombre ya que por su implementación son extremadamente frágiles dando lugar a débitos dobles, saldos negativos, modificaciones en el valor de la moneda al momento de la transacción, posibilidad de cambio de rate durante una transacción, inexistencia de rollback, etc. Decidimos atacar los más críticos que no modificaban demasiado el comportamiento actual agregando un chequeo atómico de saldo negativo con redis, apoyándonos en la arquitectura distribuida, y un chequeo de rate esperado para que no haya diferencias entre lo que el cliente esperaba y lo que recibe. Siguiendo por la línea de la seguridad agregamos algunas mejoras sin casi costo que permiten estar más protegidos contra ataques DoS lo cual termina siendo útil ya que en esta versión inicial al menos la arquitectura cuenta con tan solo un servidor de nginx.

Sacando los problemas relacionados a datos que se mencionaron antes, la app a nivel performance y disponibilidad tiene un comportamiento muy destacable dado que soporta más de 3.000 req/s con niveles de latencia por debajo de los 100 ms (quitando el exchange que tiene una latencia mínima de 400 ms por el hecho de simular la interacción con un servicio externo). El problema es que este comportamiento se ve muy afectado por el

requerimiento funcional de devolver una lista interminable de cuentas en más de un endpoint lo que hace que sea muy sencillo recibir tráfico pero no escalar en cantidad de cuentas lo que propone un escenario irreal excepto que se trate de un mercado muy limitado; solamente por mencionar un dato actualmente hay alrededor de 21 mil altcoins. Habiendo finalizado el análisis y propuesto diversas tácticas para atacar varios de los problemas relevados, concluimos que el escenario ideal está compuesto de aplicarle al sistema las tácticas 2, 3 y 4 generando así un balance donde las vulnerabilidades sean mitigadas sin afectar la complejidad ni performance de forma significativa como podría haber sido agregando un sistema de lock distribuido más completo que solo el chequeo de saldos negativos, rollbacks de transacciones, etc.

Atributo de Calidad (QA)	Caso Base (Arquitectura Inicial)	Arquitectura Final (Tácticas 2, 3 y 4)	Táctica / Mecanismo Aplicado
Fiabilidad e Integridad	Inconsistencia crítica: débitos dobles, saldos negativos y contratos corruptos ante cambios de cotización.	Consistencia garantizada: 0 saldos negativos (medido en 21.026 transacciones) y rechazo 409 ante tarifas cambiadas.	Script Lua atómico en Redis (reserveFunds) y validación de expectedRate .
Disponibilidad y Robustez	Colapso total por timeouts (ETIMEDOUT) y 100% de CPU al superar las 350–500 req/s.	Disponibilidad del 99,88% bajo 3.500 req/s. Cero caídas por timeouts; degradación controlada (HTTP 429).	Rate Limiting y Connection Limiting perimetral en Nginx (limit_req / limit_conn).
Escalabilidad y Performance	Techo estructural en 1 nodo (~300 req/s). Serialización O(N) en memoria local.	Capacidad escalada a +3.000 req/s distribuyendo carga equitativamente entre 3 réplicas (<5% CPU por réplica)	Escalamiento horizontal (<i>Scale-out</i>) con balanceo Round Robin en Nginx y estado en Redis.
Observabilidad	Nula visibilidad de métricas de negocio.	Monitoreo en tiempo real de volúmenes brutos y netos por divisa y uso de recursos por réplica.	Telemetría UDP nativa hacia StatsD y paneles unificados en Grafana.